Templates Library

Shy_Vector

2026 年 4 月 9 日

目录

1 编程 **3**
1.1 代码风格 . . . 3
1.2 开局模板 . . . 3
1.3 代码调试 . . . 4
1.4 `i128` 读写 . . . 4
1.5 字符串转数值 . . . 5
1.6 数值转字符串 . . . 5

2 数学 **6**
2.1 快速幂、自动取模 . . . 6
2.2 逆元、阶乘、阶乘逆元、组合数 . . . 7
2.3 地板除、天花板除 . . . 7
2.4 扩展 Euclid 算法 . . . 8
2.5 Euler 筛、$\varphi(n)$ . . . 8
2.6 算术基本定理 . . . 9
2.7 获取所有因数 . . . 9
2.8 异或线性基 . . . 9
2.9 矩阵乘法、矩阵快速幂 . . . 11
2.10 Gauss 消元法、线性方程组解的结构 . . . 12
2.11 常见结论 . . . 13
2.12 指数表、对数表、阶乘表、模意义分数表、素数个数 $\pi(n)$ 表、最多因数个数 $D(n)$ 表 . . . 14

3 数据结构 **15**
3.1 并查集 . . . 15
3.2 带权并查集 . . . 15
3.3 可持久化并查集 . . . 16
3.4 树状数组 . . . 18
3.5 线段树 . . . 19
3.6 线段树（动态开点） . . . 21
3.7 带懒标记的线段树 . . . 22
3.8 带懒标记的线段树（动态开点） . . . 24
3.9 可持久化线段树 . . . 25
3.10 红黑树 . . . 27

4 树 **28**
4.1 重链剖分（LCA，两点距离，祖先判断、k 级祖先） . . . 28
4.2 重链剖分（换根后的父节点、子树大小、LCA） . . . 29
4.3 重链剖分（线段树维护路径信息、子树信息） . . . 30
4.4 直径 . . . 33
4.5 重心 . . . 33

5 图 **34**
5.1 Floyd 全源最短路 . . . 34
5.2 Johnson 全源最短路 . . . 34
5.3 强连通分量缩点 . . . 36
5.4 边双连通分量缩点 . . . 37
5.5 最大流 . . . 39
5.6 最小费用最大流 . . . 40
5.7 有向图 Euler 路径/回路 . . . 42
5.8 无向图 Euler 路径/回路 . . . 43

6 串 **45**
6.1 序列 Hash（随机 base，多 mod） . . . 45
6.2 Manacher . . . 46
6.3 KMP . . . 47
6.4 Z 函数 . . . 47

7 计算几何 **48**

8 其他 **54**
8.1 整数区间二分 . . . 54
8.2 单峰离散函数最值区间 . . . 54
8.3 `std::lower_bound / std::upper_bound` . . . 55
8.4 `std::partial_sum / std::adjacent_difference` . . . 55
8.5 `std::next_permutation / std::prev_permutation` . . . 56
8.6 自定义 Hash . . . 56
8.7 使用栈模拟递归 . . . 58
8.8 位运算操作 . . . 59

1 编程

1.1 代码风格

1. 使用 **C++17** 标准，无法使用 `<bit>`、`<print>`、`this auto`、`Concept`、内部类对象返回、自定义类对象作为非类型模板参数；
2. 默认使用 **0-based 下标**，视情况使用 1-based 下标；
3. 涉及区间划分时，使用**左闭右开**区间；
4. 大括号采用 K&R 风格；
5. 缩进使用 4 空格，不使用 Tab；
6. 变量名使用下划线，类名使用大驼峰，临时类名使用下划线；
7. 词素之间使用空格间隔，遇到内容较长的下标可省略空格；
8. 不到万不得已，不要使用头文件 `<bits/stdc++.h>`；
9. 严禁使用 `using namespace std` 污染全局命名空间；
10. 除调试以外，严禁使用宏；
11. 遇到简短的代码块时，语义单一的可压行；
12. 遇到很长的代码块时，首尾和中间都使用换行，区分功能不同的代码段；
13. 函数参数过长时，换行缩进，灵活调整；
14. 善用 early return，减少不必要的缩进；
15. 使用 `std::vector` 而非全局数组；
16. 遇到 `std::vector<bool>` 时，`using u8 = uint8_t`；
17. 使用`'\n'`，而非 `std::endl`，除非遇到交互式的题目；
18. 善用 `int`、`i64`、`u64` 强调语义：下标/容量、值/答案、位运算处理；
19. 善用 `auto` 类型推导和 `ADL`，规避冗长的类型和模板参数；
20. 尽量使用 `lambda` 递归，方便捕获变量，不要使用 `std::function`；
21. 自定义比较函数时，使用 `lambda` 或自定义 `functor`，而非侵入式地重载 `operator<`；
22. 返回多个返回值时，需在内/外部自定义类型明确字段语义，不使用 `std::pair`/`std::tuple`；

1.2 开局模板

Code 1: init.cpp

```cpp
#include <cstdint>
#include <iostream>

using i64 = int64_t;

void test() {
    std::cout << "Hello World" << '\n';
}

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int T = 1;
    std::cin >> T;
    while (T--) test();
}
```

1.3 代码调试

在没有 gdb 的使用条件下，可以通过打印中间变量值进行调试.

Code 2: dbg.cpp

```cpp
namespace dbg {

    // template <class ...T>
    // std::ostream& operator<<(std::ostream& os, std::tuple<T...> const &ts) {
    //     os << '(';
    //     std::apply([&](T const &...t) { ((os << t << ','), ...); }, ts);
    //     return os << ')';
    // }

    // template <class T>
    // std::ostream& operator<<(std::ostream& os, std::vector<T> const &v) {
    //     os << '['; for (T const &x : v) os << x << ','; return os << ']';
    // }

    template <class T>
    void out(const char *s, T const &t) { std::cerr << s << '=' << t << '\n'; }

    template <class T, class ...U>
    void out(const char *s, T const &t, U const &...u) {
        while (*s != ',') std::cerr << *s++;
        std::cerr << '=' << t << ',';
        out(s + 1, u...);
    }

    #define dbg(...) dbg::out(#__VA_ARGS__, __VA_ARGS__)

}

void test() {
    int x = 114; double y = 5.14;
    auto p = std::pair{1, 2}; auto t = std::tuple{4., 5., 6.};
    std::vector<int> b{1, 2, 3, 4};
    std::vector<std::string> a{"deadbeef", "/bin/sh", "btw", "arch"};
    dbg(x, y, p, t, b); dbg(114514); dbg(a);
}
```

1.4 i128 读写

Code 3: i128.cpp

```cpp
using i128 = __int128;

std::istream &operator>>(std::istream &is, i128 &x) {
    std::string s; is >> s;
    auto it = s.begin();
    bool neg = false; x = 0;
    if (*it == '-') { neg = true; it++; }
    while (it != s.end()) x = x * 10 + (*it++ - '0');
    if (neg) x = -x;
    return is;
}

std::ostream &operator<<(std::ostream &os, i128 x) {
    if (x == 0) return os << 0;
    if (x < 0) { x = -x; os << '-'; }
    std::string s;
    while (x) { s += '0' + x % 10; x /= 10; }
    std::reverse(s.begin(), s.end());
    return os << s;
}
```

1.5 字符串转数值

使用 sto（std::string）或 strto（const char*）家族的函数.
比如转为整型 stoi(std::string s, size_t *pos, int base)，stoll，stoull：

1. 跳过空白字符（' '，'\n'，'\t'）；
2. 若 base（默认为 10）指定为 0，自动根据前缀（'+/-'，'0'，"0x"）确定；
3. 若 pos 非 nullptr，返回识别成功的开始位置至 pos（前缀不算）.

strtoll(const char *s, char **end, int base) 使用方法类似.
转为浮点数 stod(std::string s, size_t *pos)，stold，stof，支持小数点和指数'e/E'.

Code 4: stoi.cpp

```cpp
void test() {

    std::string s = "\n\t -0xff";

    size_t pos;
    int x = std::stoi(s, &pos, 0);
    assert(x == -255 && pos == 8);

    std::stoi(s); // ERROR, base == 10

}
```

1.6 数值转字符串

使用 std::to_string(T value) 将整型、浮点数转化为字符串（输出不支持进制和指数格式）.
使用 std::snprintf(char *buf, size_t n, const char* fmt, ...) 转为格式化字符串.

Code 5: itos.cpp

```cpp
void test() {

    assert(std::to_string(-0114514.1919810) == "-114514.191981");
    assert(std::to_string(-0xff) == "-255");

    char buf[32];

    std::snprintf(buf, sizeof(buf), "%X %#x %#o", 255, 255, 255);
    assert((strncmp(buf, "FF 0xff 0377", sizeof(buf)) == 0));

    std::snprintf(buf, sizeof(buf), "%06zd %07.3f", 114LL, 5.14);
    assert((strncmp(buf, "000114 005.140", sizeof(buf)) == 0));

}
```

使用 std::bitset<32>(x) 输出 x 的二进制表示.

2 数学

2.1 快速幂、自动取模

对于 $0 < P <$ INT32_MAX$/2$，Z 始终维护一个范围在 $[0, P)$ 的整数 x．对 x 进行一次加减运算后的结果会落在 $(-P, 2P)$，而 Z::norm 负责将该范围修正回 $[0, P)$．

Z 支持模意义下的四则运算、相等判断、输入输出．

Code 6: z.cpp

```cpp
struct Z {

    int x;
    Z() : x(0) {}
    Z(i64 x) : x(norm(x % p)) {}

    int val() const { return x; }
    Z inv() const { return fpow(*this, p - 2); }
    Z operator-() const { return {norm(-x)}; }
    Z& operator+=(Z b) & { x = norm(x + b.x); return *this; }
    Z& operator-=(Z b) & { x = norm(x - b.x); return *this; }
    Z& operator*=(Z b) & { x = 1LL * x * b.x % p; return *this; }
    Z& operator/=(Z b) & { return (*this) *= b.inv(); }

    friend Z operator+(Z a, Z b) { return a += b; }
    friend Z operator-(Z a, Z b) { return a -= b; }
    friend Z operator*(Z a, Z b) { return a *= b; }
    friend Z operator/(Z a, Z b) { return a /= b; }
    friend bool operator==(Z a, Z b) { return a.val() == b.val(); }
    friend bool operator!=(Z a, Z b) { return a.val() != b.val(); }

    static int p;
    static int norm(int x)
    { x += x >> 31 & p; if (x >= p) x -= p; return x; }
    static Z fpow(Z a, int b)
    { Z ans = 1; for (; b; b >>= 1, a *= a) if (b & 1) ans *= a; return ans; }

};

int Z::p = 998244353;

void test() {
    Z a = 114, b = -115;
    assert(a + b == Z::p - 1);
    assert((a *= 2).inv() == (Z{1} / 228));
}
```

2.2 逆元、阶乘、阶乘逆元、组合数

C 是动态扩容的，逆元 inv、阶乘 fact、阶乘逆元 finv.

binom(int N, int M) 返回 $\binom{N}{M} \bmod P$.

Code 7: c.cpp

```cpp
struct C {

    static int n;
    static std::vector<Z> I, F, FI;

    static Z inv(int x)  { if (x > n) resize(x << 1); return I[x]; }
    static Z fact(int x) { if (x > n) resize(x << 1); return F[x]; }
    static Z finv(int x) { if (x > n) resize(x << 1); return FI[x]; }
    static Z binom(int N, int M) { return fact(N) * finv(M) * finv(N - M); }

    static void resize(int x) {
        if (x <= n) return;
        I.resize(x + 1), F.resize(x + 1), FI.resize(x + 1);
        for (int i = n + 1; i <= x; i++) F[i] = F[i - 1] * i;
        FI[x] = F[x].inv(), I[x] = FI[x] * F[x - 1];
        for (int i = x - 1; i >= n + 1; i--)
            FI[i] = FI[i + 1] * (i + 1), I[i] = FI[i] * F[i - 1];
        n = x;
    }

};

int C::n;
std::vector<Z> C::I{0}, C::F{1}, C::FI{1};

void test() {
    assert(C::inv(2) == 499122177); // Z::p = 998244353;
    assert(C::fact(6) == 720);
    assert(C::finv(3) == Z{6}.inv());
    assert(C::binom(6, 3) == 20);
}
```

2.3 地板除、天花板除

在 C/C++ 中，a / b 如何取整，取决于编译器的具体实现．但从 C99 和 C++11 起，标准规定：商向零取整，即直接舍弃小数部分．a % b 被定义为 a - a / b，因此取模结果的符号取决于/如何取整．

Code 8: int-div.cpp

```cpp
i64 floor_div(i64 x, i64 y) {
    if (y < 0) x = -x, y = -y;
    if (x < 0) return (x - y + 1) / y;
    return x / y;
}

i64 ceil_div(i64 x, i64 y) {
    if (y < 0) x = -x, y = -y;
    if (x < 0) return x / y;
    return (x + y - 1) / y;
}
```

2.4 扩展 Euclid 算法

设 $a,b\in\mathbb{Z}$，且 $a^2+b^2>0$，欲寻找 $\gcd(a,b)$ 其中一种线性组合表示．考虑从

$$\begin{cases}1\cdot a+0\cdot b=a\\0\cdot a+1\cdot b=b\end{cases}$$

开始，对其增广矩阵

$$\begin{pmatrix}1&0&a\\0&1&b\end{pmatrix}$$

进行有限次初等行变换，让每次初等行变换对应于增广列的辗转相除法的每一步即可：

$$\cdots\to\begin{pmatrix}s&t&g\\m&n&h\end{pmatrix}\to\begin{pmatrix}m&n&h\\s-qm&t-qn&g-qh\end{pmatrix}\to\cdots\to\begin{pmatrix}k&l&\gcd(a,b)\\K&L&0\end{pmatrix}$$

Code 9: exgcd.cpp

```cpp
// x = x0 + b/d * t
// y = y0 - a/d * t
void exgcd(i64 a, i64 b, i64 &x, i64 &y, i64 &d) {
    i64 M[2][3] = {{1, 0, a}, {0, 1, b}};
    i64 *cur, *pre;
    for (cur = M[0], pre = M[1]; pre[2]; std::swap(cur, pre)) {
        if (std::abs(cur[2]) >= std::abs(pre[2])) {
            i64 q = cur[2] / pre[2];
            cur[0] -= q * pre[0];
            cur[1] -= q * pre[1];
            cur[2] -= q * pre[2];
        }
    }
    x = cur[0], y = cur[1], d = cur[2];
    if (d < 0) x = -x, y = -y, d = -d;
}
```

2.5 Euler 筛、$\varphi(n)$

`minp[x]` 是 `x` 的最小素因子，`prime` 是所有素数，`phi[x]` 是 $\varphi(x)$.

Code 10: euler.cpp

```cpp
std::vector<int> minp, prime;
// std::vector<int> phi;

void euler_sieve(int n) {
    minp.assign(n + 1, 0);
    prime.clear();
    // phi.assign(n + 1, 0);
    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            prime.push_back(i);
            // phi[i] = i - 1;
        }
        for (int p : prime) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (p == minp[i]) {
                // phi[i * p] = phi[i] * p;
                break;
            }
            // phi[i * p] = phi[i] * phi[p];
        }
    }
}
```

2.6 算术基本定理

使用前需获取素数表 `prime`，返回的 `ind` 存放着每个素因子对应的指数.

比如 $120000 = 2^6 \times 3^1 \times 5^4$，则 `ind` 存放着 `[(2, 6), (3, 1), (5, 4)]`.

Code 11: decomp.cpp

```cpp
auto decomp(i64 x) {
    std::map<int, int> ind;
    if (x == 1) return ind;
    for (int p : prime) {
        if (1LL * p * p > x) break;
        int k = 0;
        while (x % p == 0) k++, x /= p;
        if (k > 0) ind[p] = k;
    }
    if (x > 1) ind[x] = 1;
    return ind;
}
```

2.7 获取所有因数

使用前需实现算术基本定理 `decomp`，返回的 `div` 存放着所有因数.

比如 120000 的因数有 $1, 2, 2^2, \cdots, 2^6 \times 3^1 \times 5^4$，一共有 $7 \times 2 \times 5 = 70$ 个因数.

Code 12: divisor.cpp

```cpp
auto devisor(i64 x) {
    std::vector<int> div;
    div.push_back(1);
    auto ind = decomp(x);
    for (auto &[p, k] : ind) {
        int sz = div.size();
        i64 pi = 1;
        for (int i = 0; i < k; i++) {
            pi *= p;
            for (int j = 0; j < sz; j++) {
                div.push_back(div[j] * pi);
            }
        }
    }
    return div;
}
```

2.8 异或线性基

设 X 为有限维线性空间 V 中的向量集，我们可以构造基底 B 使得 X 中的每个向量 x 都可被线性表出. 由于 $\operatorname{rank} B \le \dim V = n$，基向量的个数非常少，这为我们提供了便利.

在布尔域线性空间的例子中，$\mathbb{Z}_2$ 的加法是异或 $\oplus$，乘法是与 &，可以证明 $\mathbb{Z}_2$ 是域，$V = (\mathbb{Z}_2^n, \oplus, \&, \mathbb{Z}_2)$ 是线性空间. 线性表出 x 就是选择 B 的子集，使异或和等于 x.

Code 13: xor-basis.cpp

```cpp
#include <algorithm>
#include <array>
#include <bitset>
#include <iostream>
#include <iterator>
#include <optional>
#include <vector>

template <class Key, int width, class Comp = std::less<Key>>
struct xor_basis {
```

```
    std::array<std::bitset<width>, width> basis;
    std::array<std::vector<Key>, width> keys;

    xor_basis() : basis{}, keys{} {}

    void insert(Key k, std::bitset<width> v) {
        std::vector<Key> S, temp;
        for (int i = width - 1; i >= 0; i--) {
            if (!v.test(i)) continue;
            if (basis[i].test(i)) {
                v ^= basis[i];
                std::set_symmetric_difference(
                    S.begin(), S.end(), keys[i].begin(), keys[i].end(),
                    std::back_inserter(temp), Comp{});
                S = std::move(temp);
            } else {
                basis[i] = std::move(v);
                keys[i] = std::move(S);
                return;
            }
        }
    }

    auto get_basis(std::bitset<width> v) -> std::optional<std::bitset<width>> {
        std::bitset<width> result;
        for (int i = width - 1; i >= 0; i--) {
            if (!v.test(i)) continue;
            if (basis[i].none()) return std::nullopt;
            v ^= basis[i];
            result.set(i, true);
        }
        return result;
    }

    auto get_keys(std::bitset<width> v) -> std::optional<std::vector<Key>> {
        std::vector<Key> result, temp;
        auto coord = get_basis(v);
        if (!coord) return std::nullopt;
        for (int i = width - 1; i >= 0; i--) {
            if (!coord.value().test(i)) continue;
            std::set_symmetric_difference(
                result.begin(), result.end(), keys[i].begin(), keys[i].end(),
                std::back_inserter(temp), Comp{});
            result = std::move(temp);
        }
        return result;
    }

};

void test() {
    int n;
    std::cin >> n;
    xor_basis<int, 50> B;
    for (int i = 0; i < n; i++) {
        u64 x;
        std::cin >> x;
        B.insert(i, x);
    }

    // subset of max-xor
    std::bitset<50> ans = 0;
    for (int i = 49; i >= 0; i--) {
        if (B.basis[i].none()) continue;
        if (B.basis[i][i] ^ ans[i]) ans ^= B.basis[i];
    }
    std::cout << ans.to_ullong() << '\n';
}
```

2.9 矩阵乘法、矩阵快速幂

Grid 实现普通二维数组的功能，Matrix 实现矩阵的功能，使用 CRTP 技术实现不同数据类型的矩阵.

Code 14: matrix.cpp

```cpp
template <class T>
struct Grid {
    int n, m;
    std::vector<std::vector<T>> A;
    Grid(int n) : Grid(n, n) {}
    Grid(int n, int m) : n(n), m(m), A(n, std::vector<T>(m)) {}
    Grid(std::vector<std::vector<T>> const& B) : n(B.size()), m(B[0].size()), A(B) {}
    auto operator[](int i) -> std::vector<T> & { return A[i]; }
    auto operator[](int i) const -> std::vector<T> const& { return A[i]; }
};

template <class T, class Mat>
struct Matrix : Grid<T> {

    using Grid<T>::n, Grid<T>::m, Grid<T>::A, Grid<T>::Grid;
    explicit Matrix(int n) : Grid<T>(n) { for (int i = 0; i < n; i++) A[i][i] = 1; }

    Mat operator*(Mat const& B) const {
        auto that = static_cast<Mat const*>(this);
        Mat C = *that; return C *= B;
    }

    Mat& operator*=(Mat const& B) {
        auto that = static_cast<Mat*>(this);
        Mat C(n, B.m);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++)
                for (int j = 0; j < B.m; j++)
                    C[i][j] += A[i][k] * B[k][j];
        return *that = std::move(C);
    }

    Mat pow(i64 k) const {
        auto that = static_cast<Mat const*>(this);
        Mat result(n), base(*that);
        for (; k; k >>= 1, base *= base) if (k & 1) result *= base;
        return result;
    }

};

struct FMat : Matrix<double, FMat> {
    using Base = Matrix<double, FMat>;
    using Base::Base;
};

struct ZMat : Matrix<Z, FMat> {
    using Base = Matrix<Z, FMat>;
    using Base::Base;
};
```

2.10 Gauss 消元法、线性方程组解的结构

解 $Ax = b$. 若得特解 $Ap = b$，则任取 $v \in \ker A$，总有 $A(p + v) = b$，因此方程解的结构是 $x = p + v$.

`solve(A, b)` 返回特解 p 和右零空间 $\ker A$ 的基底 B.

特别地，当 $\operatorname{rank} A < \operatorname{rank} [A \quad b]$ 时，方程无解；当 $\operatorname{rank} A = n$ （A 的列数）时，方程有唯一解.

Code 15: gauss-elim.cpp

```cpp
void gauss(FMat& A, FMat& B) {

    int n = A.n, m = A.m;

    auto find_pivot = [&](int r, int c) -> std::optional<int> {
        for (int i = r; i < n; i++) if (std::abs(A[i][c]) > eps) return i;
        return std::nullopt;
    };

    auto normalize = [&](int r, int c) {
        for (int j = c + 1; j < m; j++) A[r][j] /= A[r][c];
        for (int j = 0; j < B[0].size(); j++) B[r][j] /= A[r][c];
        A[r][c] = 1;
    };

    auto eliminate = [&](int r, int c) {
        for (int i = 0; i < n; i++) {
            if (i == r) continue;
            for (int j = c + 1; j < m; j++) A[i][j] -= A[i][c] * A[r][j];
            for (int j = 0; j < B[0].size(); j++) B[i][j] -= A[i][c] * B[r][j];
            A[i][c] = 0;
        }
    };

    int r = 0, c = 0;
    while (r < n && c < m) {
        auto p = find_pivot(r, c);
        if (!p) { c++; continue; }
        std::swap(A[p.value()], A[r]);
        std::swap(B[p.value()], B[r]);
        normalize(r, c);
        eliminate(r, c);
        r++, c++;
    }

}

auto solve(FMat A, FMat b) -> std::optional<std::pair<FMat, FMat>> {
    gauss(A, b);
    int n = A.n, m = A.m, r = 0, c = 0;
    std::vector<int> pivot, is_pivot(m);
    while (r < n && c < m) {
        if (A[r][c]) {
            pivot.push_back(c);
            is_pivot[c] = true;
            r++;
        }
        c++;
    }

    for (int i = r; i < n; i++)
        if (std::abs(b[i][0]) > eps)
            return std::nullopt;

    // dim V = rank A + dim null A
    //     m =      r +    (m - r)
    // [ ]   [ ]     [  ]     [  ]             [    ]
    // [x] = [p] + t1[b1] + t2[b2] + ... + tm-r[bm-r]
    // [ ]   [ ]     [  ]     [  ]             [    ]

    FMat particular(m, 1), nullspace_basis(m, m - r);
```

```
    for (int i = 0; i < r; i++) particular[pivot[i]][0] = b[i][0];
    for (int j = 0, cnt = 0; j < m; j++) {
        if (is_pivot[j]) continue;
        for (int i = 0; i < r; i++)
            nullspace_basis[pivot[i]][cnt] = -A[i][j];
        nullspace_basis[j][cnt++] = 1;
    }
    return {{particular, nullspace_basis}};
}

void test() {
    int n;
    std::cin >> n;
    FMat A(n, n), b(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) std::cin >> A[i][j];
        std::cin >> b[i][0];
    }
    auto solution = solve(A, b);
    if (!solution) return std::cout << -1 << '\n', void(); // no solution
    auto [p, B] = solution.value();
    if (!B[0].empty()) return std::cout << 0 << '\n', void(); // many solutions
    for (int i = 0; i < p.n; i++)
        std::cout << 'x' << i + 1 << '=' << p[i][0] << '\n'; // unique solution
}
```

2.11 常见结论

位运算：

- $a \mid b = (a \,\&\, b) + (a \oplus b)$；
- $a + b = (a \oplus b) + 2(a \,\&\, b) = (a \mid b) + (a \,\&\, b)$；
- $a \,\&\, b \le \min(a, b) \le \max(a, b) \le a \mid b$；
- $a + b - 2\min(a, b) = |a - b| \le a \oplus b \le a \mid b \le a + b$；
- $\gcd(a, b) \le \min(a, b, a - b, a \bmod b)$，$a > b$；
- $\bigoplus_{k=1}^{n} k = n, 1, n + 1, 0 \quad (n \bmod 4 = 0, 1, 2, 3)$.

$\varphi(n)$：

- $\sum_{k=1}^{n} [k \perp n]k = n\varphi(n)/2$；
- $\sum_{d \mid n} \varphi(d) = n$，$\sum_{d \mid n} \mu(d)(n/d) = \varphi(n)$；
- $\sum_{k=1}^{n} \gcd(k, n) = \sum_{d \mid n} \lfloor n/d \rfloor \varphi(d)$；

组合数：

- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$，$k\binom{n}{k} = n\binom{n-1}{k-1}$，$\binom{n}{m}\binom{m}{k} = \binom{n}{k}\binom{n-k}{m-k}$；
- $\sum_{k=0}^{n} \binom{n}{k} = 2^n$，$\sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0$；
- $\sum_{k=0}^{n} k\binom{n}{k} = n \cdot 2^{n-1}$，$\sum_{k=0}^{n} k^2 \binom{n}{k} = n(n-1) \cdot 2^{n-2}$；
- $\sum_{k=1}^{n} \frac{1}{k}\binom{n}{k} = \sum_{k=1}^{n} \frac{1}{k}$；
- $\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$.

Fibonacci 数列：

- $F_a \mid F_b$，当且仅当 $a \mid b$；
- $\gcd(F_a, F_b) = F_{\gcd(a,b)}$.

2.12 指数表、对数表、阶乘表、模意义分数表、素数个数 $\pi(n)$ 表、最多因数个数 $D(n)$ 表

素数个数 $\pi(n) \sim \frac{n}{\ln n}$，因数最多的数的因数个数 $D(n)$ 的级别大约是 $O(\sqrt[3]{n})$.

k	2^k
10	1,024
16	65,536
20	1,048,576
31	2,147,483,648
32	4,294,967,296
48	281,474,976,710,656
63	9,223,372,036,854,775,808
64	18,446,744,073,709,551,616

n	$\log_2 n$	n	$\log_2 n$
3	1.585	10^5	16.610
5	2.322	2×10^5	17.610
7	2.807	5×10^5	18.932
9	3.170	10^6	19.932
10	3.322	10^8	26.575
10^2	6.644	10^{12}	39.863
10^3	9.966	9×10^{18}	62.965
10^4	13.288	18×10^{18}	63.965

n	7	8	9	10	11	12	20
$n!$	5,040	40,320	362,880	3,628,800	39,916,800	479,001,600	2,432,902,008,176,640,000

a	b	$a/b \bmod 998,244,353$	$a/b \bmod 1,000,000,007$
1	2	499,122,177	500,000,004
1	3	332,748,118	333,333,336
1	4	748,683,265	250,000,002
1	5	598,946,612	400,000,003
1	6	166,374,059	166,666,668
1	7	855,638,017	142,857,144
1	8	873,463,809	125,000,001
1	9	443,664,157	111,111,112
2	3	665,496,236	666,666,672
2	5	199,648,871	800,000,006
2	7	713,031,681	285,714,288
2	9	887,328,314	222,222,224
3	4	249,561,089	750,000,006
3	5	798,595,483	200,000,002
3	7	570,425,345	428,571,432
3	8	623,902,721	375,000,003
4	5	399,297,742	600,000,005
4	7	427,819,009	571,428,576
4	9	776,412,275	444,444,448
5	6	831,870,295	833,333,340
5	7	285,212,673	714,285,720
5	8	374,341,633	625,000,005
5	9	221,832,079	555,555,560
6	7	142,606,337	857,142,864
7	8	124,780,545	875,000,007
7	9	110,916,040	777,777,784
8	9	554,580,197	888,888,896

n	$\le n$ 的素数个数
10	4
10^2	25
10^3	168
10^4	1,229
10^5	9,592
10^6	78,498
10^7	664,579
10^8	5,761,455
10^9	50,847,534
10^{12}	37,607,912,018
10^{18}	24,739,954,287,740,860

n	最多因数个数
10	4
10^2	25
10^3	32
10^4	64
10^5	128
10^6	240
10^7	448
10^8	768
10^9	1,344
10^{12}	6,720
10^{18}	103,680

3 数据结构

3.1 并查集

节点编号从 0 到 $n-1$，使用了路径压缩 + 启发式合并的实现，单次操作平均时间复杂度为 $O(\alpha(n))$.

Code 16: dsu.cpp

```cpp
struct DSU {

    std::vector<int> fa, sz;

    DSU(int n) { init(n); }

    void init(int n) {
        fa.resize(n);
        std::iota(fa.begin(), fa.end(), 0);
        sz.assign(n, 1);
    }

    int find(int x) {
        int t, r = x;
        while (fa[r] != r) r = fa[r];
        while (fa[x] != x) t = fa[x], fa[x] = r, x = t;
        return r;
    }

    bool same(int x, int y) { return find(x) == find(y); }
    bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (sz[x] > sz[y]) std::swap(x, y);
        fa[x] = y, sz[y] += sz[x];
        return true;
    }

};
```

3.2 带权并查集

$x, y \in X$ 不仅有关系，还带权 $f(x,y)$，满足**自反性** $f(x,x) = 1_X$，并且存在运算 $\circ$，使得权值 f 满足**传递性**

$$f(x,z) = f(x,y) \circ f(y,z)$$

只要存在运算 $\neg$，使得权值 f 满足**对称性**

$$f(y,x) = \neg f(x,y)$$

那么 f 可以使用带权并查集来维护.

设 x, y 的根分别为 r_x, r_y，已知 $f(x,y), f(x,r_x), f(y,r_y)$，欲使 $r_{r_x} = r_y$，有

$$f(r_x, r_y) = \neg f(x,r_x) \circ f(x,y) \circ f(y,r_y)$$

由于自反性，如果合并时已经有 $r_x = r_y$，那么右式必等于 1_X，利用这一点来判断关系 f 是否良定义. 在这之后的适当时机（查询时），修正 $f(x,r_x) \mapsto f(x,r_x) \circ f(r_x,r_y)$.

最经典的 f 是数轴上两点的有向距离（如区间和），也可以是环上的两点距离（如“食物链”的模 n 加法群）. 下面以异或为例：自反性 $x \oplus x = 0$，传递性 $x \oplus z = (x \oplus y) \oplus (y \oplus z)$，对称性 $y \oplus x = x \oplus y$，使用带权并查集维护：

Code 17: w-dsu.cpp

```cpp
struct DSU {

    std::vector<int> fa, sz;
    std::vector<int> xor_with_root; // f(x, rx)

    DSU(int n) { init(n); }

    void init(int n) {
        fa.resize(n);
        std::iota(fa.begin(), fa.end(), 0);
        sz.assign(n, 1);
        xor_with_root.assign(n, 0); // x ^ x = 0
    }

    int find(int x) {
        if (fa[x] == x) return x;
        int root = find(fa[x]); // fa[fa[x]] == new_rt
        xor_with_root[x] ^= xor_with_root[fa[x]]; // fa[x] == old_rt
        return fa[x] = root; // fa[x] == new_rt
    }

    bool merge(int x, int y, int w) {
        int rx = find(x), ry = find(y);
        // f(rx, ry) = ~f(x, rx) * f(x, y) * f(y, ry)
        //           =  f(x, rx) * f(x, y) * f(y, ry)
        w = xor_with_root[x] ^ w ^ xor_with_root[y];
        if (rx == ry) return w == 0;
        // f(ry, rx) = ~f(rx, ry) =  f(rx, ry)
        if (sz[rx] > sz[ry]) std::swap(rx, ry), w = w;
        fa[x] = y, sz[y] += sz[x];
        xor_with_root[x] = w;
        return true;
    }

};
```

3.3 可持久化并查集

可持久化并查集是 leafy 的.

对 `fa` 和 `sz` 数组实现了可持久化，单次访问 $O(\log n)$.

对两棵树启发式合并，保证了树高是 $O(\log n)$ 级别，因此一次查询是 $O(\log^2 n)$ 的.

1. `PDSU()`：初始化从节点 0 到节点 $n-1$ 的并查集作为第 0 个版本，m 是对最大版本数的估计；
2. `latest()/latest_()`：返回最近版本号/根节点；
3. `save()/save_()`：传入版本号/根节点，拷贝该版本作为新版本；
4. `find()/find_()`：传入版本号/根节点、索引 `i`，返回同类的代表（叶）节点以及对应的索引；
5. `same()/same_()`：传入版本号/根节点、两个索引，判定是否同类；
6. `merge()/merge_()`：传入版本号/根节点、两个索引，合并后**自动产生新版本**；
7. `merged()/merged_()`：传入版本号/根节点、两个索引，合并后**返回新根，不产生新版本**；
8. `leaf()/leaf_()`：传入版本号/根节点、索引 `i`，返回对应的叶节点；
9. `modify()/modify_()`：传入版本号/根节点、待修改节点的索引 `i`、修改内容（`fa` 和 `sz`），返回新根.

Code 18: p-dsu.cpp

```cpp
struct PDSU {

    int n;
    std::vector<int> fa, sz;
    std::vector<int> root, lson, rson;

    PDSU(int n, int m) : n(n) {
        int z = n * 4 + m * (32 - __builtin_clz(n));
        root.reserve(m), lson.reserve(z), rson.reserve(z);
        fa.reserve(z), sz.reserve(z);
        root.push_back(build(0, n));
    }

    int node(int lch, int rch, int f, int z) {
        lson.push_back(lch), rson.push_back(rch);
        fa.push_back(f), sz.push_back(z);
        return lson.size() - 1;
    }

    int build(int l, int r) {
        if (l + 1 == r) return node(-1, -1, l, 1);
        int m = (l + r) >> 1;
        return node(build(l, m), build(m, r), -1, 0);
    }

    int latest() { return root.size() - 1; }
    int latest_() { return root.back(); }

    void save(int v) { save_(root[v]); }
    void save_(int rt) { root.push_back(rt); }

    int leaf(int v, int i) { return leaf_(root[v], i); }
    int leaf_(int rt, int i) { return leaf_(rt, 0, n, i); }
    int leaf_(int rt, int l, int r, int i) {
        if (l + 1 == r) return rt;
        int m = (l + r) >> 1;
        if (i < m) return leaf_(lson[rt], l, m, i);
        return leaf_(rson[rt], m, r, i);
    }

    int modify(int v, int i, int f, int z) { return modify_(root[v], i, f, z); }
    int modify_(int rt, int i, int f, int z) { return modify_(rt, 0, n, i, f, z); }
    int modify_(int rt, int l, int r, int i, int f, int z) {
        if (l + 1 == r) return node(-1, -1, f, z);
        int m = (l + r) >> 1;
        if (i < m) return node(modify_(lson[rt], l, m, i, f, z), rson[rt], -1, 0);
        return node(lson[rt], modify_(rson[rt], m, r, i, f, z), -1, 0);
    }

    struct find_result_t { int leaf, index; };
    auto find(int v, int i) -> find_result_t { return find_(root[v], i); }
    auto find_(int rt, int i) -> find_result_t {
        int p = leaf_(rt, i);
        while (fa[p] != i) i = fa[p], p = leaf_(rt, i);
        return {p, i};
    }

    bool same(int v, int i, int j) { return find(v, i).index == find(v, j).index; }
    bool same_(int rt, int i, int j) { return find_(rt, i).index == find_(rt, j).index; }

    void merge(int v, int i, int j) { merge_(root[v], i, j); }
    void merge_(int rt, int i, int j) { save_(merged_(rt, i, j)); }
    int merged(int v, int i, int j) { return merged_(root[v], i, j); }
    int merged_(int rt, int i, int j) {
        auto [p, x] = find_(rt, i);
        auto [q, y] = find_(rt, j);
        if (x == y) return rt;
        if (sz[p] > sz[q]) std::swap(p, q), std::swap(x, y);
```

```
        return modify_(modify_(rt, x, y, sz[p]), y, y, sz[p] + sz[q]);
    }

};

void test() {
    PDSU dsu(100, 10);                          // v0
    dsu.merge(dsu.latest(), 1, 2);              // v1
    dsu.merge(dsu.latest(), 2, 3);              // v2
    dsu.merge(0, 1, 3);                         // v3
    int rt = dsu.latest_();                     // v3
    dsu.merge_(rt, 3, 4);                       // v4
    assert(dsu.same(dsu.latest(), 1, 4));
    int v = dsu.latest();
    rt = dsu.merged(v, 4, 5);                   // temp
    rt = dsu.merged_(rt, 5, 6);                 // temp
    rt = dsu.merged_(rt, 6, 7);                 // temp
    dsu.save_(rt);                              // v5
    dsu.save(dsu.latest());                     // v6 = v5
    assert(dsu.find(dsu.latest(), 1).index
        == dsu.find(dsu.latest(), 7).index);
    dsu.save(3);                                // v7 = v3
    assert(!dsu.same(dsu.latest(), 2, 3));
}
```

3.4 树状数组

数组元素 $a_i \in A$，对 a 交替进行：**单点修改** $a_i \mapsto a_i * x$；**前缀查询** $S(k) = \prod_{0 \le i < k} a_i$.

如果 $(A, *)$ 构成**半群** (即 $*$ 仅满足结合律)，那么可以使用树状数组 p 来维护 a：

$$p_k = \prod_{k-\text{lowbit}(k) \le i < k} a_i, \quad 1 \le k \le n$$

每个 p_k 管辖了在 a_k 前面 $\text{lowbit}(k)$ 个元素. 在实践中，为了方便建树，我们额外要求 $(A, *)$ 具备**幺元**，即幺半群.

前缀查询时，$S(k)$ 始终能被拆成 $O(\log k)$ 个 p_i 之积，以达成 $O(\log n)$ 的查询复杂度，常数因子非常优秀. 具体来说，$S(k)$ 能按照 k 的二进制表示，被拆成 $\text{popcount}(k)$ 个 p_i 之积. 例如 $k = 22$ 的情形：

$$(10110)_2 = (10000)_2 + (00100)_2 + (00010)_2$$
$$[0, 22) = [0, 16) \cup [16, 20) \cup [20, 22)$$
$$S(22) = p_{16} * p_{20} * p_{22}$$

单点修改时，只需修改 $O(\log n)$ 个管辖范围包含 a_i 的区间. 修改区间从 p_{i+1} 开始，能够证明 p_k 的下一个管辖范围包含 a_i 的区间是 $p_{k+\text{lowbit(k)}}$.

Code 19: fenwick.cpp

```
template <class T, class F>
struct Fenwick {

    int n; F f;
    std::vector<T> p;
    Fenwick(F f) : f(f) {}

    void build(std::vector<T> const &a) {
        n = a.size();
        p.assign(n + 1, {});
        for (int x = 1; x <= n; x++) {
            p[x] = f(p[x], a[x - 1]);
            int y = x + (x & -x);
            if (y > n) continue;
            p[y] = f(p[y], p[x]);
        }
    }
```

```
18
19      // i \in [0, n)
20      void modify(int i, T const &v) {
21          for (int x = i + 1; x <= n; x += x & -x)
22              p[x] = f(p[x], v);
23      }
24
25      // return S[0, k), where k \in [1, n]
26      T query(int k) {
27          T ans = T{};
28          for (int x = k; x >= 1; x -= x & -x)
29              ans = f(ans, p[x]);
30          return ans;
31      }
32
33      // return r \in [1, n], where S[0, r) is the largest of <= v
34      int find(T const &v) {
35          T acc{}; int r = 0, width = 32 - __builtin_clz(n);
36          for (int mask = 1 << (width - 1); mask; mask >>= 1) {
37              if ((r | mask) > n) continue;
38              T tmp = f(acc, p[r | mask]);
39              if (tmp > v) continue;
40              r |= mask;
41              acc = tmp;
42          }
43      }
44
45  };
46
47  void test() {
48      int n, m;
49      std::cin >> n >> m;
50      std::vector<i64> arr(n);
51      for (auto &x : arr) std::cin >> x;
52
53      using add = std::plus<i64>;
54      Fenwick<i64, add> fw(add{});
55      fw.build(arr);
56
57      while (m--) {
58          int op;
59          std::cin >> op;
60          if (op == 1) {
61              int i; i64 v;
62              std::cin >> i >> v; i--;
63              fw.modify(i, v);
64          } else if (op == 2) {
65              int l, r;
66              std::cin >> l >> r; l--, r--, r++;
67              std::cout << fw.query(r) - fw.query(l) << '\n';
68          }
69      }
70  }
```

3.5 线段树

今有半群 $(D, +)$. 和树状数组一样，线段树能维护由**半群**元素构成的数组，比如区间和/积、区间最值、区间 gcd. 相比树状数组使用二进制结构维护区间片段以合成前缀区间，线段树直接二分区间直至区间长度为 1，因此可以实现任意的**区间查询**. 如果只有**单点修改**的需求，可以不用实现懒标记.

在实践中，为了方便合并子区间信息，我们额外要求 $(D, +)$ 还具备**幺元**，即幺半群.

1. `Info::Info()`：D 的幺元；
2. `Info::operator+(Info)`：D 的运算 $+$.

使用时：

1. build 要求在建树前完成对 info 的初始化 init；
2. modify 单点赋值：对 info[i] 赋值；
3. query 区间查询：对 info[L, R) 信息汇总.

Code 20: sgt.cpp

```cpp
template <class Info>
struct SGT {

    int n;
    std::vector<Info> info;

    void build(std::vector<Info> const &init) {
        auto build_ = [&](auto &f, int p, int l, int r) -> void {
            if (l + 1 == r) return info[p] = init[l], void();
            int m = (l + r) >> 1;
            f(f, p << 1, l, m);
            f(f, p << 1 | 1, m, r);
            info[p] = info[p << 1] + info[p << 1 | 1];
        };
        n = init.size();
        info.resize(n << 2);
        build_(build_, 1, 0, n);
    }

    void modify(int i, Info const &v) { modify(1, 0, n, i, v); }
    void modify(int p, int l, int r, int i, Info const &v) {
        if (l + 1 == r) return info[p] = v, void();
        int m = (l + r) >> 1;
        if (i < m) modify(p << 1, l, m, i, v);
        else modify(p << 1 | 1, m, r, i, v);
        info[p] = info[p << 1] + info[p << 1 | 1];
    }

    Info query(int i) { return query(i, i + 1); }
    Info query(int L, int R) { return query(1, 0, n, L, R); }
    Info query(int p, int l, int r, int L, int R) {
        if (R <= l || r <= L) return {};
        if (L <= l && r <= R) return info[p];
        int m = (l + r) >> 1;
        return query(p << 1, l, m, L, R) + query(p << 1 | 1, m, r, L, R);
    }

};

void test() {

    struct Info {
        i64 S;
        Info() : S(0) {}
        Info(i64 S) : S(S) {}
        Info operator+(Info const &rhs) { return {S + rhs.S}; }
    };

    int n, m;
    std::cin >> n >> m;

    std::vector<Info> arr(n);
    for (auto &[S] : arr) std::cin >> S;
    SGT<Info> sgt;
    sgt.build(arr);

    while (m--) {
        int op;
        std::cin >> op;
        if (op == 1) {
            int i; i64 v;
            std::cin >> i >> v; i--;
            sgt.modify(i, sgt.query(i) + Info{v});
```

```
        } else if (op == 2) {
            int l, r;
            std::cin >> l >> r; l--, r--, r++;
            std::cout << sgt.query(l, r).S << '\n';
        }
    }

}
```

3.6 线段树（动态开点）

当区间分布跨度极广，但区间总长在可接受范围内时，可以用动态开点的方式代替直接在极广区间上建树．此时父子节点的编号关系已不存在，需要额外维护每个节点的 `lson` 和 `rson`．每次从根出发向下查询，需要时刻 `push` 探路，没路就新开节点 `node`．

使用 `i64` 表示区间端点．动态开点的线段树**与普通线段树使用方法完全一致**，能访问的下标得到极大扩展．使用前需给定全局最广区间 `[mini, maxi)`，但请注意：**越广的区间意味着越大的常数**．

Code 21: sgt-dynamic.cpp

```
template <class Info>
struct SGT {

    i64 mini, maxi;
    std::vector<int> lson, rson;
    std::vector<Info> info;

    SGT(i64 mini, i64 maxi) : mini(mini), maxi(maxi), lson(2), rson(2), info(2) {}

    int node() {
        lson.emplace_back(); rson.emplace_back(); info.emplace_back();
        return info.size() - 1;
    }

    void pull(int p) { info[p] = info[lson[p]] + info[rson[p]]; }
    void push(int p) { if (!lson[p]) lson[p] = node(); if (!rson[p]) rson[p] = node(); }

    void modify(i64 i, Info const &v) { modify(1, mini, maxi, i, v); }
    void modify(int p, i64 l, i64 r, i64 i, Info const &v) {
        if (l + 1 == r) return info[p] = v, void();
        i64 m = l + (r - l) / 2;
        push(p);
        if (i < m) modify(lson[p], l, m, i, v);
        else modify(rson[p], m, r, i, v);
        pull(p);
    }

    Info query(i64 i) { return query(i, i + 1); }
    Info query(i64 L, i64 R) { return query(1, mini, maxi, L, R); }
    Info query(int p, i64 l, i64 r, i64 L, i64 R) {
        if (R <= l || r <= L) return {};
        if (L <= l && r <= R) return info[p];
        i64 m = l + (r - l) / 2;
        push(p);
        return query(lson[p], l, m, L, R) + query(rson[p], m, r, L, R);
    }

};

void test() {
    int n, m;
    std::cin >> n >> m;

    struct Info {...};
    SGT<Info> sgt(-1e18, 1e18);
    for (int i = 0; i < n; i++) {
        i64 x;
```

```
        std::cin >> x;
        sgt.modify(i, {x});
    }

    while (m--) {...}
}
```

3.7 带懒标记的线段树

形式化地，信息 $(D,+)$ 构成**半群**，标记 $(T,*)$ 构成**幺半群**，另外定义 $\cdot : T \times D \to D$，并且 $\cdot$ 对 $+$ 满足**分配律**，这种代数结构我们可以使用带懒标记的线段树来维护. 为了方便合并子区间信息，我们额外要求 $(D,+)$ 还具备幺元，即幺半群.

使用前需定义 `Info` 和 `Tag`，包括：

1. `Info::Info()`：D 的幺元；
2. `Info::operator+(Info)`：D 的运算 $+$；
3. `Info::operator+=(Tag)`：满足分配律的运算 $\cdot$；
4. `Tag::Tag()`：T 的幺元；
5. `Tag::operator+=(Tag)`：T 的运算 $*$.

使用时：

1. `build` 要求在建树前完成对 `info` 的初始化 `init`；
2. `modify` 单点赋值：对 `info[i]` 赋值；
3. `apply` 区间操作：对 `info[L, R)` 施加操作 `t`；
4. `query` 区间查询：对 `info[L, R)` 信息汇总.

Code 22: sgt-tag.cpp

```
template <class Info, class Tag>
struct SGT {

    int n;
    std::vector<Info> info;
    std::vector<Tag> tag;

    void apply(int p, Tag const &t) { info[p] += t; tag[p] += t; }
    void pull(int p) { info[p] = info[p << 1] + info[p << 1 | 1]; }
    void push(int p) { apply(p << 1, tag[p]), apply(p << 1 | 1, tag[p]); tag[p] = {}; }

    void build(std::vector<Info> const &init) {
        auto build_ = [&](auto &f, int p, int l, int r) -> void {
            if (l + 1 == r) return info[p] = init[l], void();
            int m = (l + r) >> 1;
            f(f, p << 1, l, m);
            f(f, p << 1 | 1, m, r);
            pull(p);
        };
        n = init.size();
        info.assign(n << 2, {});
        tag.assign(n << 2, {});
        build_(build_, 1, 0, n);
    }

    void modify(int i, Info const &v) { modify(1, 0, n, i, v); }
    void modify(int p, int l, int r, int i, Info const &v) {
        if (l + 1 == r) return info[p] = v, void();
        int m = (l + r) >> 1;
        push(p);
        if (i < m) modify(p << 1, l, m, i, v);
        else modify(p << 1 | 1, m, r, i, v);
```

```
        pull(p);
    }

    void apply(int L, int R, Tag const &t) { apply(1, 0, n, L, R, t); }
    void apply(int p, int l, int r, int L, int R, Tag const &t) {
        if (R <= l || r <= L) return;
        if (L <= l && r <= R) return apply(p, t);
        int m = (l + r) >> 1;
        push(p);
        apply(p << 1, l, m, L, R, t);
        apply(p << 1 | 1, m, r, L, R, t);
        pull(p);
    }

    Info query(int i) { return query(i, i + 1); }
    Info query(int L, int R) { return query(1, 0, n, L, R); }
    Info query(int p, int l, int r, int L, int R) {
        if (R <= l || r <= L) return {};
        if (L <= l && r <= R) return info[p];
        int m = (l + r) >> 1;
        push(p);
        return query(p << 1, l, m, L, R) + query(p << 1 | 1, m, r, L, R);
    }

};

void test() {

    int n, q, m;
    std::cin >> n >> q >> m;
    Z::p = m;

    // tag: ax + b
    // tag -> tag: A(ax+b)+B = (Aa)x+(Ab+B)
    // tag -> info: sum(ax+b) = (a)sum(x)+(nb)

    struct Tag {
        Z a, b;
        Tag() : a(1), b(0) {}
        Tag(Z a, Z b) : a(a), b(b) {}
        void operator+=(Tag const &t) { a *= t.a; b *= t.a; b += t.b; }
    };

    struct Info {
        Z S;
        int l, r;
        Info() : S(0), l(0), r(0) {}
        Info(Z S, int l, int r) : S(S), l(l), r(r) {}
        Info operator+(Info const &b) { return {S + b.S, l, b.r}; }
        void operator+=(Tag const &t) { S *= t.a; S += (r - l) * t.b; }
    };

    std::vector<Info> arr(n);
    for (int i = 0; i < n; i++) {
        i64 x;
        std::cin >> x;
        arr[i] = {x, i, i + 1};
    }

    SGT<Info, Tag> sgt;
    sgt.build(arr);
    while (q--) {
        int op, x, y;
        std::cin >> op >> x >> y; x--; y--; y++;
        if (op == 1) {
            i64 k;
            std::cin >> k;
            sgt.apply(x, y, {k, 0});
        } else if (op == 2) {
            i64 k;
```

```
            std::cin >> k;
            sgt.apply(x, y, {1, k});
        } else if (op == 3) {
            std::cout << sgt.query(x, y).S.val() << '\n';
        }
    }

}
```

3.8 带懒标记的线段树（动态开点）

当区间分布跨度极广，但区间总长在可接受范围内时，可以用动态开点的方式代替直接在极广区间上建树．此时父子节点的编号关系已不存在，需要额外维护每个节点的 `lson` 和 `rson`．每次从根出发向下查询，需要时刻 `push` 探路，没路就新开节点 `node`．

使用 `i64` 表示区间端点．动态开点的线段树**与普通线段树使用方法完全一致**，能访问的下标得到极大扩展．使用前需给定全局最广区间 `[mini, maxi)`，但请注意：**越广的区间意味着越大的常数**．

Code 23: sgt-tag-dynamic.cpp

```
template <class Info, class Tag>
struct SGT {

    i64 mini, maxi;
    std::vector<int> lson, rson;
    std::vector<Info> info;
    std::vector<Tag> tag;

    SGT(i64 mini, i64 maxi)
        : mini(mini), maxi(maxi), lson(2), rson(2), info(2), tag(2) {}

    int node() {
        lson.emplace_back(); rson.emplace_back();
        info.emplace_back(); tag.emplace_back();
        return info.size() - 1;
    }

    void apply(int p, Tag const &t) { info[p] += t; tag[p] += t; }
    void pull(int p) { info[p] = info[lson[p]] + info[rson[p]]; }
    void push(int p) {
        if (!lson[p]) lson[p] = node();
        if (!rson[p]) rson[p] = node();
        apply(lson[p], tag[p]);
        apply(rson[p], tag[p]);
        tag[p] = {};
    }

    void modify(i64 i, Info const &v) { modify(1, mini, maxi, i, v); }
    void modify(int p, i64 l, i64 r, i64 i, Info const &v) {
        if (l + 1 == r) return info[p] = v, void();
        i64 m = l + (r - l) / 2;
        push(p);
        if (i < m) modify(lson[p], l, m, i, v);
        else modify(rson[p], m, r, i, v);
        pull(p);
    }

    void apply(i64 L, i64 R, Tag const &t) { apply(1, mini, maxi, L, R, t); }
    void apply(int p, i64 l, i64 r, i64 L, i64 R, Tag const &t) {
        if (r <= L || R <= l) return;
        if (L <= l && r <= R) return apply(p, t);
        i64 m = l + (r - l) / 2;
        push(p);
        apply(lson[p], l, m, L, R, t);
        apply(rson[p], m, r, L, R, t);
        pull(p);
```

```
    }

    Info query(i64 i) { return query(i, i + 1); }
    Info query(i64 L, i64 R) { return query(1, mini, maxi, L, R); }
    Info query(int p, i64 l, i64 r, i64 L, i64 R) {
        if (R <= l || r <= L) return {};
        if (L <= l && r <= R) return info[p];
        i64 m = l + (r - l) / 2;
        push(p);
        return query(lson[p], l, m, L, R) + query(rson[p], m, r, L, R);
    }

};

void test() {
    int n, q, m;
    std::cin >> n >> q >> m;
    Z::p = m;

    struct Tag {...};
    struct Info {...};
    SGT<Info, Tag> sgt(0, n);
    for (int i = 0; i < n; i++) {
        i64 x;
        std::cin >> x;
        sgt.modify(i, {x, i, i + 1});
    }

    while (q--) {...}
}
```

3.9 可持久化线段树

仅支持**单点修改**，接口和前面可持久化并查集类似，支持手动控制版本保存：

1. `build()`：初始化第 0 个版本，m 是对最大版本数的估计；
2. `latest()/latest_()`：返回最近版本号/根节点；
3. `save()/save_()`：传入版本号/根节点，拷贝该版本作为新版本；
4. `modify()/modify_()`：传入版本号/根节点，单点修改，**自动保存版本**；
5. `modified()/modified_()`：传入版本号/根节点，单点修改，**不保存版本**，返回新根以供用户自由处理；
6. `query()/query_()`：传入版本号/根节点，区间查询，不产生版本.

Code 24: p-sgt.cpp

```
template <class Info>
struct PSGT {

    int n;
    std::vector<int> root, lson, rson;
    std::vector<Info> info;

    int node() {
        lson.emplace_back();
        rson.emplace_back();
        info.emplace_back();
        return info.size() - 1;
    }

    int clone(int p) {
        lson.push_back(lson[p]);
        rson.push_back(rson[p]);
        info.push_back(info[p]);
        return info.size() - 1;
    }
```

```
21
22      void build(std::vector<Info> const &init, int m) {
23          auto build_ = [&](auto &f, int l, int r) -> int {
24              int p = node();
25              if (l + 1 == r) return info[p] = init[l], p;
26              int m = (l + r) >> 1;
27              lson[p] = f(f, l, m);
28              rson[p] = f(f, m, r);
29              info[p] = info[lson[p]] + info[rson[p]];
30              return p;
31          };
32          node();
33          n = init.size();
34          root.reserve(m);
35          int z = n * 4 + m * (32 - __builtin_clz(n));
36          lson.reserve(z), rson.reserve(z), info.reserve(z);
37          root.push_back(build_(build_, 0, n));
38      }
39
40      int latest() { return root.size() - 1; }
41      int latest_() { return root.back(); }
42      void save(int ver) { save_(root[ver]); }
43      void save_(int p) { root.push_back(p); }
44
45      void modify(int ver, int i, Info const &v) { save_(modified(ver, i, v)); }
46      void modify_(int p, int i, Info const &v) { save_(modified_(p, i, v)); }
47
48      int modified(int ver, int i, Info const &v) { return modified_(root[ver], i, v); }
49      int modified_(int p, int i, Info const &v) { return modified_(p, 0, n, i, v); }
50      int modified_(int p, int l, int r, int i, Info const &v) {
51          p = clone(p);
52          if (l + 1 == r) return info[p] = v, p;
53          int m = (l + r) >> 1;
54          if (i < m) lson[p] = modified_(lson[p], l, m, i, v);
55          else rson[p] = modified_(rson[p], m, r, i, v);
56          info[p] = info[lson[p]] + info[rson[p]];
57          return p;
58      }
59
60      Info query(int ver, int L, int R) { return query_(root[ver], L, R); }
61      Info query_(int p, int L, int R) { return query_(p, 0, n, L, R); }
62      Info query_(int p, int l, int r, int L, int R) {
63          if (r <= L || R <= l) return {};
64          if (L <= l && r <= R) return info[p];
65          int m = (l + r) >> 1;
66          return query_(lson[p], l, m, L, R) + query_(rson[p], m, r, L, R);
67      }
68
69  };
70
71  void test() {
72
73      int n, m;
74      std::cin >> n >> m;
75
76      struct Info {
77          i64 S;
78          Info() : S(0) {}
79          Info(i64 S) : S(S) {}
80          Info operator+(Info const &rhs) { return {S + rhs.S}; }
81      };
82
83      PSGT<Info> sgt;
84      std::vector<Info> arr(n);
85      for (auto &[S] : arr) std::cin >> S;
86      sgt.build(arr, m);
87
88      while (m--) {
89          int v, op, i;
```

```
        std::cin >> v >> op >> i; i--;
        if (op == 1) {
            i64 c;
            std::cin >> c;
            sgt.modify(v, i, {c});
        } else if (op == 2) {
            std::cout << sgt.query(v, i, i + 1).S << '\n';
            sgt.save(v);
        }
    }

}
```

3.10 红黑树

直接使用 libstdc++ 标准库里 pb_ds 的红黑树实现：

Code 25: pbds.cpp

```cpp
#include <cassert>
#include <cstdint>
#include <functional>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using i64 = int64_t;
using RBT = __gnu_pbds::tree<i64, __gnu_pbds::null_type, std::less<i64>,
                __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>;

void test() {

    RBT tr;
    auto [it, ok] = tr.insert(114);
    tr.insert(514), tr.insert(19), tr.insert(19), tr.insert(810);

    assert(tr.empty() == false);
    assert(tr.size() == 5);

    // [19, 19, 114, 514, 810]
    assert(tr.erase(1919) == false);
    assert(*(it = tr.erase(it)) == 514);
    assert(it + 2 == tr.end());
    assert(tr.erase(tr.end()) == tr.end());

    // [19, 19, 514, 810]
    it = tr.find_by_order(2); assert(*it == 514);
    int pos = tr.order_of_key(486); assert(pos == 2);

    // tr[l, r) -> tr[0, 2) -> [19, 19]
    auto l = tr.lower_bound(19); assert(*l == 19);
    auto r = tr.upper_bound(19); assert(*r == 514);

    tr.insert(998), tr.insert(244), tr.insert(353);
    // [19, 19, 244, 353, 514, 810, 998]

    RBT tr2;
    tr.split(353, tr2);
    for (auto i = tr.begin(); i != tr.end(); i++);
    // tr: [19, 19, 244, 353], tr2: [514, 810, 998]
    tr.join(tr2);
    // tr: [19, 19, 244, 353, 514, 810, 998], tr2: []

    tr2.insert(400);
    // tr.join(tr2); // ERROR: range cross!

}
```

4 树

4.1 重链剖分（LCA，两点距离，祖先判断、k 级祖先）

经过重链剖分，一棵树被拆成若干链，链内是重边，链外是轻边．从根节点出发，每经过一次轻边，当前子树大小至少减半．因此在任意路径上，轻边数量至多 $O(\log n)$ 条，即路径总能拆成 $O(\log n)$ 段连续链区间．

关于 dfn 序（0-based）的性质：

- 一条链上自上而下，dfn 构成递增的连续区间，可据此在链上寻址、对路径操作；
- 子树所有节点的 dfn 构成的集合是连续区间，可据此对子树操作．

我们使用：

- `top[u]` 表示 `u` 所处链的头部；
- `in[u]` 表示 `u` 的 dfn；
- `[in[u], out[u])` 表示 `u` 子树的 dfn 区间；
- `seq[i]` 将 dfn 序 `i` 转化为对应节点编号．

节点编号是 0-based 的．

Code 26: hld-base.cpp

```cpp
struct HLD {

    int n;
    std::vector<int> fa, sz, top, dep, in, out, seq;
    std::vector<std::vector<int>> adj;

    HLD(int n) : n(n), fa(n), sz(n), top(n), dep(n), in(n), out(n), seq(n), adj(n) {}
    void add_edge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }

    void build(int root) {
        top[root] = root, dep[root] = 0, fa[root] = -1;
        dfs1(root), dfs2(root);
    }

    void dfs1(int u) {
        if (fa[u] != -1) {
            auto it = std::find(adj[u].begin(), adj[u].end(), fa[u]);
            adj[u].erase(it);
        }
        sz[u] = 1;
        for (auto& v : adj[u]) {
            fa[v] = u;
            dep[v] = dep[u] + 1;
            dfs1(v);
            sz[u] += sz[v];
            auto& h = adj[u].front();
            if (sz[v] > sz[h]) std::swap(v, h);
        }
    }

    void dfs2(int u) {
        static int dfn = 0;
        in[u] = dfn++;
        seq[in[u]] = u;
        for (auto v : adj[u]) {
            if (v == adj[u].front()) top[v] = top[u];
            else top[v] = v;
            dfs2(v);
        }
        out[u] = dfn;
    }

    int lca(int u, int v) {
        while (top[u] != top[v]) {
```

```
            if (dep[top[u]] > dep[top[v]]) u = fa[top[u]];
            else v = fa[top[v]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    int leap(int u, int k) {
        int d = dep[u] - k;
        if (d < 0) return -1;
        while (dep[top[u]] > d) u = fa[top[u]];
        return seq[in[u] + d - dep[u]];
    }

    int dist(int u, int v) { return dep[u] + dep[v] - dep[lca(u, v)] * 2; }
    bool is_ancestor(int anc, int ch) { return in[anc] <= in[ch] && in[ch] < in[anc]; }

};

void test() {
    int n, m, s;
    std::cin >> n >> m >> s; s--;

    HLD tr(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        std::cin >> u >> v; u--, v--;
        tr.add_edge(u, v);
    }
    tr.build(s);

    while (m--) {
        int a, b;
        std::cin >> a >> b; a--, b--;
        std::cout << tr.lca(a, b) + 1 << '\n';
    }
}
```

4.2 重链剖分（换根后的父节点、子树大小、LCA）

`get_rooted_fa` 和 `get_rooted_sz` 的复杂度都是 $O(\log \deg p)$.

Code 27: hld-rooted.cpp

```
struct HLD {

    ...

    int get_rooted_fa(int root, int p) {
        if (p == root) return p;
        if (!is_ancestor(p, root)) return fa[p];
        auto dfn_order = [&] (int u, int v) { return in[u] < in[v]; };
        auto it = std::upper_bound(adj[p].begin(), adj[p].end(), root, dfn_order) - 1;
        return *it;
    }

    int get_rooted_sz(int root, int p) {
        if (p == root) return n;
        if (!is_ancestor(p, root)) return sz[p];
        return n - sz[get_rooted_fa(root, p)];
    }

    int get_rooted_lca(int a, int b, int c) {
        return lca(a, b) ^ lca(b, c) ^ lca(c, a);
    }

};
```

4.3 重链剖分（线段树维护路径信息、子树信息）

使用线段树维护 dfn 区间.

Code 28: hld-sgt.cpp

```cpp
template <class Info, class Tag>
struct HLD {

    ...

    SGT<Info, Tag> sgt;

    HLD(int n) ... {}
    void add_edge(int u, int v) {...}

    void build(int root) {...}
    void build_sgt(std::vector<Info> const &init) {
        std::vector<Info> init_(n);
        for (int i = 0; i < n; i++) init_[in[i]] = init[i];
        sgt.build(init_);
    }

    void dfs1(int u) {...}
    void dfs2(int u) {...}

    void modify(int u, const Info &v) { sgt.modify(in[u], v); }

    void subtree_apply(int u, const Tag &t) { sgt.apply(in[u], out[u], t); }
    Info subtree_query(int u) { return sgt.query(in[u], out[u]); }

    void path_apply(int u, int v, const Tag &t) {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) std::swap(u, v);
            sgt.apply(in[top[u]], in[u] + 1, t);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) std::swap(u, v);
        sgt.apply(in[u], in[v] + 1, t);
    }

    Info undirected_path_query(int u, int v) {
        Info u2v;
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) std::swap(u, v);
            u2v = u2v + sgt.query(in[top[u]], in[u] + 1);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) std::swap(u, v);
        return u2v + sgt.query(in[u], in[v] + 1);
    }

};

void test() {

    int N, M, R;
    std::cin >> N >> M >> R >> Z::p; R--;
    std::vector<i64> a(N);
    for (auto &x : a) std::cin >> x;

    struct Tag {
        Z z;
        Tag() : z(0) {}
        Tag(Z z) : z(z) {}
        void operator+=(Tag const &t) { z += t.z; }
    };

    struct Info {
        int l, r; Z S;
```

```
        Info() : l(0), r(0), S(0) {}
        Info(int l, int r, Z S) : l(l), r(r), S(S) {}
        void operator+=(Tag const &t) { S += (r - l) * t.z; }
        Info operator+(Info const &o) { return {l, o.r, S + o.S}; }
    };

    HLD<Info, Tag> hld(N); // step 0: num of nodes
    for (int i = 0; i < N - 1; i++) {
        int u, v;
        std::cin >> u >> v; u--; v--;
        hld.add_edge(u, v); // step 1: add edges
    }
    hld.build(R); // step 2: dfs twice
    std::vector<Info> init(N);
    for (int i = 0; i < N; i++) init[i] = {hld.in[i], hld.in[i] + 1, a[i]}; // in[] !!!
    hld.build_sgt(init); // step 3: init info, build sgt

    while (M--) {
        int op;
        std::cin >> op;
        if (op == 1) {
            int x, y; i64 z;
            std::cin >> x >> y >> z; x--; y--;
            hld.path_apply(x, y, {z});
        } else if (op == 2) {
            int x, y;
            std::cin >> x >> y; x--; y--;
            std::cout << hld.undirected_path_query(x, y).S.val() << '\n';
        } else if (op == 3) {
            int x; i64 z;
            std::cin >> x >> z; x--;
            hld.subtree_apply(x, {z});
        } else if (op == 4) {
            int x;
            std::cin >> x; x--;
            std::cout << hld.subtree_query(x).S.val() << '\n';
        }
    }

}
```

如果要维护有向的 dfn 区间（比如点权的结合（如仿射函数的复合）不满足交换律），那么需要为 `Info` 设计正反两向区间信息 `Info::forw/backw`. 合并 `Info` 时，**正向信息正向拼接，反向信息反向拼接.**

Code 29: hld-sgt-directed.cpp

```
template <class Info, class Tag>
struct HLD {

    ...

    Info directed_path_query(int u, int v) {
        Info r2u, r2v;
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                r2u = sgt.query(in[top[u]], in[u] + 1) + r2u;
                u = fa[top[u]];
            } else {
                r2v = sgt.query(in[top[v]], in[v] + 1) + r2v;
                v = fa[top[v]];
            }
        }
        if (dep[u] > dep[v]) r2u = sgt.query(in[v], in[u] + 1) + r2u;
        else r2v = sgt.query(in[u], in[v] + 1) + r2v;
        return {r2u.backw * r2v.forw, r2v.backw * r2u.forw};
    }

};

```

```cpp
struct LinearFunc {
    i64 k, b;
    i64 operator()(i64 x) { return k * x + b; }
};

struct DirectedInfo {
    LinearFunc f;
    DirectedInfo() : f(1, 0) {}
    DirectedInfo operator*(DirectedInfo const &o) {
        auto &[k1, b1] = f;
        auto &[k2, b2] = o.f;
        // f1 * f2 != f2 * f1
        // (f1 * f2)(.) = f2(f1(.)) = k2 (k1 x + b1) + b2
        return {LinearFunc{k1 * k2, b1 * k2 + b2}};
    }
};

struct Tag {...};

struct Info {
    DirectedInfo forw, backw;

    Info() : forw(), backw() {}
    Info(DirectedInfo forw, DirectedInfo backw): forw(forw), backw(backw) {}

    void operator+=(Tag const &t) {...}
    Info operator+(Info const &o) { return {l, o.r, forw * o.forw, o.backw * backw}; }
};
```

4.4 直径

经过子树根节点的直径由最深路径和次深路径组成，使用树形 DP 解决，支持负边权，时间复杂度为 $O(n)$.

Code 30: diameter.cpp

```cpp
int d = 0;
auto dfs = [&] (auto &f, int u, int fa) -> void {
    for (int v : adj[u]) {
        if (v == fa) continue;
        f(f, v, u);
        d = std::max(d, dp[u] + dp[v] + 1); // w(u, v) = 1
        dp[u] = std::max(dp[u], dp[v] + 1); // w(u, v) = 1
    }
};
```

4.5 重心

去除重心后，每棵子树大小不超过 $n/2$，据此寻找重心. 求有根树每棵子树的重心，时间复杂度为 $O(n)$.

Code 31: centroid.cpp

```cpp
std::vector<int> fa(n), sz(n), w(n), cent(n);
auto pull = [&](auto &f, int u, int fa_) -> void {
    fa[u] = fa_, sz[u] = 1;
    for (int v : adj[u]) {
        if (v == fa) continue;
        f(f, v, u);
        sz[u] += sz[v];
        w[u] = std::max(w[u], sz[v]);
    }
    cent[u] = u;
    for (int v : adj[u]) {
        if (v == fa) continue;
        for (int p = cent[v]; p != u; p = fa[p]) {
            if (std::max(w[p], sz[u] - sz[p]) <= sz[u] / 2) {
                cent[u] = p;
                break;
            }
        }
    }
}
```

5 图

5.1 Floyd 全源最短路

Floyd 适合在稠密图上使用，时间复杂度为 $O(N^3)$.

Code 32: floyd.cpp

```cpp
template <class T>
auto floyd(std::vector<std::vector<T>> const &w) {
    auto d = w; int n = w.size();
    for (int k = 0; k < n; k++) {
        for (int u = 0; u < n; u++) {
            for (int v = 0; v < n; v++) {
                d[u][v] = std::min(d[u][v], d[u][k] + d[k][v]);
            }
        }
    }
    return d;
}

void test() {
    int n, m;
    std::cin >> n >> m;

    auto w = std::vector(n, std::vector(n, INT64_MAX / 2));
    for (int u = 0; u < n; u++) w[u][u] = 0;
    for (int i = 0; i < m; i++) {
        int u, v; i64 f;
        std::cin >> u >> v >> f; u--, v--;
        w[u][v] = std::min(w[u][v], f);
        w[v][u] = std::min(w[v][u], f);
    }

    auto d = floyd(w);
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            std::cout << d[u][v] << " \n"[v == n - 1];
        }
    }
}
```

5.2 Johnson 全源最短路

先从超级源点开始跑一遍 SPFA 获取所有点的最短距离作为势能 $h(u)$，然后把势能差加在边权上 $w'(u,v) = w(u,v) + h(u) - h(v)$ 使其非负，最后从每个点出发跑 Dijkstra（使用优先队列优化），减去 $h(u) - h(v)$ 即可获得原始距离. Johnson 适合在稀疏图上使用，时间复杂度为 $O(NM \log M)$.

当原图存在负环时，`johnson()` 返回为空；当两点之间不存在路径时，距离值为 `Graph<T>::INF`.

Code 33: johnson.cpp

```cpp
template <class T>
struct Graph {

    int n;
    std::vector<T> dist, h;
    std::vector<int> len, inq;
    std::vector<std::vector<int>> adj;
    static constexpr T INF = std::numeric_limits<T>::max() / 2;

    struct Edge { int u, v; T w; };
    std::vector<Edge> e;

    Graph(int n) : n(n), adj(n) {}
    void add_edge(int u, int v, T w) {
        adj[u].push_back(e.size());
```

```
        e.push_back({u, v, w});
    }

    bool spfa() {
        std::queue<int> q;
        len.assign(n, 0), inq.assign(n, true), h.assign(n, 0);
        for (int u = 0; u < n; u++) q.push(u);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            inq[u] = false;
            for (int i : adj[u]) {
                auto [_, v, w] = e[i];
                if (h[u] + w < h[v]) {
                    h[v] = h[u] + w;
                    len[v] = len[u] + 1;
                    if (len[v] > n) return false;
                    if (!inq[v]) inq[v] = true, q.push(v);
                }
            }
        }
        return true;
    }

    void dijkstra(int s) {
        using state = std::pair<T, int>;
        std::priority_queue<state, std::vector<state>, std::greater<>> q;
        dist.assign(n, INF), dist[s] = 0, q.emplace(0, s);
        while (!q.empty()) {
            auto [d, u] = q.top(); q.pop();
            if (d > dist[u]) continue;
            for (int i : adj[u]) {
                auto [_, v, w] = e[i];
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    q.emplace(dist[v], v);
                }
            }
        }
    }

    auto johnson() {
        std::vector<std::vector<T>> ans;
        if (!spfa()) return ans;
        for (auto &[u, v, w] : e) w += h[u] - h[v];
        for (int u = 0; u < n; u++) {
            dijkstra(u);
            ans.push_back(dist);
            for (int v = 0; v < n; v++) {
                if (dist[v] != INF) {
                    ans[u][v] -= h[u] - h[v];
                }
            }
        }
        return ans;
    }

};

void test() {
    int n, m;
    std::cin >> n >> m;
    Graph<i64> g(n);
    for (int i = 0; i < m; i++) {
        int u, v; i64 w;
        std::cin >> u >> v >> w; u--, v--;
        g.add_edge(u, v, w);
    }
    auto dist = g.johnson();
    if (dist.empty()) { std::cout << -1 << '\n'; return; }
```

```
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            if (dist[u][v] == Graph<i64>::INF) {
                std::cout << u << ' ' << v << '\n';
            }
        }
    }
}
```

5.3 强连通分量缩点

有向图上两点彼此可达，则称该点对强连通．强连通是等价关系，因此可将有向图上的点划分为若干强连通分量，通常会将同一个分量下的点缩成一点．

节点编号是 0-based 的，建图使用 `add_edge(u, v)`，使用 `compress()` 缩点后：

- `SCC::cid`：原图上的点所处强连通分量的编号（0-based）；
- `SCC::Graph::n`：缩点后的点数，即强连通分量个数；
- `SCC::Graph::dots[i]`：强连通分量 `i` 内的所有节点；
- `SCC::Graph::edges`：缩点后的边集，端点是强连通分量的编号．

缩点后的图是有向无环图（DAG）．

Code 34: scc.cpp

```
struct SCC {

    int n;
    std::vector<int> cid;
    std::vector<std::vector<int>> adj;
    SCC(int n) : n(n), adj(n) {}
    void add_edge(int u, int v) { adj[u].push_back(v); }

    auto tarjan() {
        std::vector<std::vector<int>> scc;
        cid.assign(n, -1);

        int timer = 0;
        std::stack<int> stk;
        std::vector<int> dfn(n, -1), low(n, -1);

        auto dfs = [&](auto &f, int u) -> void {
            low[u] = dfn[u] = timer++;
            stk.push(u);
            for (int v : adj[u]) {
                if (dfn[v] == -1) f(f, v), low[u] = std::min(low[u], low[v]);
                else if (cid[v] == -1) low[u] = std::min(low[u], dfn[v]);
            }
            if (low[u] == dfn[u]) {
                int id = scc.size();
                scc.push_back({});
                while (true) {
                    int v = stk.top(); stk.pop();
                    scc.back().push_back(v);
                    cid[v] = id;
                    if (v == u) break;
                }
            }
        };

        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
                dfs(dfs, i);
            }
        }
```

```
41
42          return scc;
43      }
44
45      struct Graph {
46          std::vector<std::vector<int>> dots;
47          std::vector<std::pair<int, int>> edges;
48      };
49
50      Graph compress() {
51          auto scc = tarjan();
52          Graph G{std::move(scc), {}};
53          for (int u = 0; u < n; u++) {
54              for (int v : adj[u]) {
55                  if (cid[u] == cid[v]) continue;
56                  G.edges.emplace_back(cid[u], cid[v]);
57              }
58          }
59          return G;
60      }
61
62  };
63
64  void test() {
65      int n, m;
66      std::cin >> n >> m;
67      SCC g(n);
68      for (int i = 0; i < m; i++) {
69          int u, v;
70          std::cin >> u >> v; u--, v--;
71          g.add_edge(u, v);
72      }
73      auto [dots, edges] = g.compress();
74      std::cout << dots.size() << '\n';
75      for (auto &cc : dots) {
76          std::cout << cc.size() << ' ';
77          for (int u : cc) std::cout << u + 1 << ' ';
78          std::cout << '\n';
79      }
80  }
```

5.4 边双连通分量缩点

删除无向图上任意一边，两点始终连通，则称该点对边双连通．与强连通分量缩点相似，边双连通也是等价关系，可以将同一分量上的点缩为一点．在无向图的 DFS 树中，向 dfn 小的点的连边不再有横跨边，只有返祖边．当 `low == dfn` 时，说明子树向祖先的路径必经过根所在的桥，形成边双连通分量．

在原图上可以通过检查 `cid` 是否相同来判断是否为桥．

可处理重边和自环．缩点后的图是由桥边组成的森林．

Code 35: edcc.cpp

```
1   struct EDCC {
2
3       int n;
4       std::vector<int> cid;
5       std::vector<std::vector<std::array<int, 2>>> adj;
6       EDCC(int n) : n(n), adj(n) {}
7
8       void add_edge(int u, int v) {
9           static int eid = 0;
10          adj[u].push_back({eid, v});
11          adj[v].push_back({eid, u});
12          eid += 1;
13      }
14
15      auto tarjan() {
```

```
        std::vector<std::vector<int>> edcc;
        cid.assign(n, -1);

        int timer = 0;
        std::stack<int> stk;
        std::vector<int> dfn(n, -1), low(n, -1);

        auto dfs = [&](auto &f, int u, int in) -> void {
            low[u] = dfn[u] = timer++;
            stk.push(u);
            for (auto [eid, v] : adj[u]) {
                if (eid == in) continue;
                if (dfn[v] == -1) {
                    f(f, v, eid);
                    low[u] = std::min(low[u], low[v]);
                } else if (cid[v] == -1 && dfn[v] < dfn[u]) {
                    low[u] = std::min(low[u], dfn[v]);
                }
            }
            if (low[u] == dfn[u]) {
                int id = edcc.size();
                edcc.push_back({});
                while (true) {
                    int v = stk.top(); stk.pop();
                    edcc.back().push_back(v);
                    cid[v] = id;
                    if (v == u) break;
                }
            }
        };

        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
                dfs(dfs, i, -1);
            }
        }

        return edcc;
    }

    struct Graph {
        std::vector<std::vector<int>> dots;
        std::vector<std::pair<int, int>> edges;
    };

    Graph compress() {
        auto edcc = tarjan();
        Graph G{std::move(edcc), {}};
        for (int u = 0; u < n; u++) {
            for (auto [eid, v] : adj[u]) {
                if (cid[u] < cid[v]) {
                    G.edges.emplace_back(cid[u], cid[v]);
                }
            }
        }
        return G;
    }

};

void test() {
    int n, m;
    std::cin >> n >> m;
    EDCC g(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        std::cin >> u >> v; u--, v--;
        g.add_edge(u, v);
    }
```

```
    auto [dots, edges] = g.compress();
    std::cout << dots.size() << '\n';
    for (auto &cc : dots) {
        std::cout << cc.size() << ' ';
        for (int u : cc) std::cout << u + 1 << ' ';
        std::cout << '\n';
    }
}
```

5.5 最大流

Dinic 算法使用 BFS 分层避免过大的增广深度，多路同时增广，当前弧优化跳过了此轮增广已拒流的弧．获得最大流后，自动获得当前的最小割：由最大流最小割定理，两者相等．

- `MaxFlow::add_edge(u, v, cap)`：添加带容量 `cap` 的有向边 `i`，自动添加反向边 `i ^ 1` 实现反悔；
- `MaxFlow::flow(s, t)`：BFS 分层，DFS 多路增广，直至无法增广；
- `MaxFlow::get_flow(u)`：获取该有向边的当前流量；
- `MaxFlow::get_side(u)`：判断该点是否位于最小割的源点一侧．

使用 `get_flow` 和 `get_side` 之前，都需要先调用 `flow` 完成最大流．

Code 36: max-flow.cpp

```
template <class T>
struct MaxFlow {

    int n;
    std::vector<int> dep, cur;
    std::vector<std::vector<int>> adj;

    struct Edge { int to; T cap; };
    std::vector<Edge> e;

    MaxFlow(int n) : n(n), adj(n) {}
    void add_edge(int u, int v, T cap) {
        adj[u].push_back(e.size());
        e.push_back({v, cap});
        adj[v].push_back(e.size());
        e.push_back({u, 0});
    }

    T dfs(int u, int t, T flow) {
        if (u == t) return flow;
        T resi = flow;
        for (int& timer = cur[u]; timer < adj[u].size(); timer++) {
            int eid = adj[u][timer];
            auto [v, c] = e[eid];
            if (dep[v] != dep[u] + 1) continue;
            if (c <= 0) continue;
            T subf = dfs(v, t, std::min(resi, c));
            e[eid].cap -= subf, e[eid^1].cap += subf;
            resi -= subf;
            if (resi <= 0) break;
        }
        return flow - resi;
    }

    bool bfs(int s, int t) {
        std::queue<int> q;
        dep.assign(n, -1);
        dep[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int eid : adj[u]) {
```

```
                auto [v, c] = e[eid];
                if (dep[v] != -1) continue;
                if (c <= 0) continue;
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        return dep[t] != -1;
    }

    T flow(int s, int t) {
        T acc = 0;
        while (bfs(s, t)) {
            cur.assign(n, 0);
            acc += dfs(s, t, std::numeric_limits<T>::max());
        }
        return acc;
    }

    T get_flow(int eid) { return e[eid << 1 | 1].cap; }
    bool get_side(int u) { return dep[u] != -1; }

};

void test() {
    int n, m, s, t;
    std::cin >> n >> m >> s >> t; s--, t--;
    MaxFlow<i64> g(n);
    for (int i = 0; i < m; i++) {
        int u, v; i64 w;
        std::cin >> u >> v >> w; u--, v--;
        g.add_edge(u, v, w);
    }
    std::cout << g.flow(s, t) << '\n';
}
```

5.6 最小费用最大流

SSP 算法适用于无负圈网络，每次贪心选择最短的增广路.

由于正反两向边费用互为相反数，存在负边权，无法直接使用 Dijkstra 获得增广路. Primal-Dual 原始对偶算法仿照 Johnson 全源最短路算法，为每个点赋予势能，使新边权非负. 每轮增广后图的形态发生变化，用最短距离叠加到势能上更新势能.

`MinCostFlow` 的使用方法与 `MaxFlow` 相同.

Code 37: min-cost-flow.cpp

```
template <class T>
struct MinCostFlow {

    int n;
    std::vector<T> dist, h;
    std::vector<int> trace;
    std::vector<std::vector<int>> adj;
    static constexpr T INF = std::numeric_limits<T>::max() / 2;

    struct Edge { int to; T cap, cost; };
    std::vector<Edge> e;

    MinCostFlow(int n) : n(n), adj(n) {}
    void add_edge(int u, int v, T cap, T cost) {
        adj[u].push_back(e.size());
        e.push_back({v, cap, cost});
        adj[v].push_back(e.size());
        e.push_back({u, 0, -cost});
    }

```

```
    bool johnson(int s, int t) {
        using state = std::pair<T, int>;
        std::priority_queue<state, std::vector<state>, std::greater<>> q;
        dist.assign(n, INF), trace.assign(n, -1);
        dist[s] = 0, q.emplace(0, s);
        while (!q.empty()) {
            auto [d, u] = q.top(); q.pop();
            if (d > dist[u]) continue;
            for (int i : adj[u]) {
                auto [v, cap, cost] = e[i];
                if (cap > 0 && d + cost + h[u] - h[v] < dist[v]) {
                    dist[v] = d + cost + h[u] - h[v];
                    trace[v] = i;
                    q.emplace(dist[v], v);
                }
            }
        }
        return dist[t] != INF;
    }

    struct result { T flow, cost; };
    auto flow(int s, int t) {
        T flow = 0, cost = 0;
        h.assign(n, 0);
        while (johnson(s, t)) {
            T allow = INF;
            for (int i = 0; i < n; i++) h[i] += dist[i];
            for (int v = t; v != s; v = e[trace[v]^1].to)
                allow = std::min(allow, e[trace[v]].cap);
            for (int v = t; v != s; v = e[trace[v]^1].to)
                e[trace[v]].cap -= allow,
                e[trace[v]^1].cap += allow;
            flow += allow;
            cost += allow * h[t];
        }
        return result{flow, cost};
    }

    T get_flow(int eid) { return e[eid << 1 | 1].cap; }

};

void test() {
    int n, m, s, t;
    std::cin >> n >> m >> s >> t; s--, t--;
    MinCostFlow<i64> g(n);
    for (int i = 0; i < m; i++) {
        int u, v; i64 w, c;
        std::cin >> u >> v >> w >> c; u--, v--;
        g.add_edge(u, v, w, c);
    }
    auto [flow, cost] = g.flow(s, t);
    std::cout << flow << ' ' << cost << '\n';
}
```

5.7 有向图 Euler 路径/回路

- 迹：点可重走，边不可重走；
- Euler 路径：经过所有边的迹；
- Euler 回路：起点和终点相同的 Euler 路径.

含 N 个点的有向图存在 Euler 路径，当且仅当

- 排除孤立点后，图是弱连通的；
- 恰有 $N-2$ 个点，入度和出度相等；
- 恰有 1 个点（起点），出度比入度大 1；
- 恰有 1 个点（终点），入度比出度大 1.

特别地，零图存在 Euler 路径（空集），是半欧拉图. 但模板不为此做特判，将返回空的 `trail`.

含 N 个点的有向图存在 Euler 回路，当且仅当

- 排除孤立点后，图是弱连通的；
- 每个点的入度和出度相等.

Hierholzer 算法第一轮 DFS 终止在终点/起点，形成主线/环，从上面的点 `x` 出发继续沿着未访问的出边 DFS，回到点 `x` 终止，一直循环往复. 回溯把沿途边压入答案中，逆序输出. 时间复杂度为 $O(N+M)$.

`eulerian()` 返回沿着 Euler 路径上所有边的 `eid`，使用 `Graph::e[eid]` 获取边的信息.

使用前，可以调用 `lexico()` 使输出的 Euler 路径的点列字典序最小.

Code 38: eulerian-directed.cpp

```cpp
struct Graph {

    int n;
    std::vector<int> in, out;
    std::vector<std::vector<int>> adj;

    struct Edge { int u, v; };
    std::vector<Edge> e;

    Graph(int n) : n(n), in(n), out(n), adj(n) {}
    void add_edge(int u, int v) {
        out[u] += 1, in[v] += 1;
        adj[u].push_back(e.size()); e.push_back({u, v});
    }

    void lexico() {
        for (auto &v : adj)
            std::sort(v.begin(), v.end(),
                [&](int i, int j) { return e[i].v < e[j].v; }); }

    auto eulerian() -> std::vector<int> {
        std::vector<int> trail;
        for (int u = 0, odd = 0; u < n; u++)
            if (in[u] != out[u] && (++odd > 2
                || std::abs(in[u] - out[u]) > 1)) return trail;

        int s = -1;
        for (int u = 0; u < n; u++) if (out[u] > in[u]) { s = u; break; }
        if (s == -1) for (int u = 0; u < n; u++) if (out[u]) { s = u; break; }
        if (s == -1) return {};

        std::vector<int> cur(n);
        auto dfs = [&](auto &self, int u) -> void {
            while (cur[u] < adj[u].size()) {
                int eid = adj[u][cur[u]++];
                auto [_, v] = e[eid];
                self(self, v);
                trail.push_back(eid);
```

```
            }
        };

        dfs(dfs, s);

        if (trail.size() < e.size()) return {};
        std::reverse(trail.begin(), trail.end());
        return trail;
    }

};

void test() {
    int n, m;
    std::cin >> n >> m;
    Graph g(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        std::cin >> u >> v; u--, v--;
        g.add_edge(u, v);
    }

    g.lexico();
    auto trail = g.eulerian();
    if (trail.empty()) { std::cout << "No" << '\n'; return; }
    for (int eid : trail) { auto [u, v] = g.e[eid]; std::cout << u + 1 << ' '; }
    std::cout << g.e[trail.back()].v + 1 << '\n';
}
```

5.8 无向图 Euler 路径/回路

含 N 个点的无向图存在 Euler 路径，当且仅当

- 排除孤立点后，图是连通的；
- 恰有 $N-2$ 个点，度数是偶数；
- 恰有 2 个点（起点和终点），度数是奇数.

特别地，零图存在 Euler 路径（空集），是半欧拉图. 但模板不为此做特判，将返回空的 `trail`.

含 N 个点的无向图存在 Euler 回路，当且仅当

- 排除孤立点后，图是连通的；
- 所有点的度数都是偶数.

在实现中，无向边用正反两条有向边表示，正向的 `eid` 是偶数（0-based），反向的 `eid` 是奇数，通过 `eid ^ 1` 获取对应的反边，用于避免 DFS 重复访边. 通过 `eid >> 1` 获取对应无向边的编号（0-based）.

Code 39: eulerian-undirected.cpp

```
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <vector>

using i64 = int64_t;

struct Graph {

    int n;
    std::vector<int> deg;
    std::vector<std::vector<int>> adj;

    struct Edge { int u, v; };
    std::vector<Edge> e;

```

```
    Graph(int n) : n(n), deg(n), adj(n) {}
    void add_edge(int u, int v) {
        deg[u] += 1, deg[v] += 1;
        adj[u].push_back(e.size()); e.push_back({u, v});
        adj[v].push_back(e.size()); e.push_back({v, u});
    }

    void lexico() {
        for (auto &v : adj)
            std::sort(v.begin(), v.end(),
                [&](int i, int j) { return e[i].v < e[j].v; }); }

    auto eulerian() -> std::vector<int> {
        std::vector<int> trail;
        for (int u = 0, odd = 0; u < n; u++)
            if ((deg[u] & 1) && ++odd > 2) return trail;

        int s = -1;
        for (int u = 0; u < n; u++) if (deg[u] & 1) { s = u; break; }
        if (s == -1) for (int u = 0; u < n; u++) if (deg[u]) { s = u; break; }
        if (s == -1) return {};

        std::vector<int> cur(n), vis(e.size());
        auto dfs = [&](auto &self, int u) -> void {
            while (cur[u] < adj[u].size()) {
                int eid = adj[u][cur[u]++];
                if (vis[eid ^ 1]) continue;
                vis[eid] = true;
                auto [_, v] = e[eid];
                self(self, v);
                trail.push_back(eid);
            }
        };

        dfs(dfs, s);

        if (trail.size() < e.size() / 2) return {};
        std::reverse(trail.begin(), trail.end());
        return trail;
    }

};

void test() {
    int N, M;
    std::cin >> N >> M;
    Graph g(N);
    for (int i = 0; i < M; i++) {
        int u, v;
        std::cin >> u >> v;
        g.add_edge(u, v);
    }

    g.lexico();
    auto trail = g.eulerian();
    if (trail.empty() && !g.e.empty()) { std::cout << "No" << '\n'; return; }

    std::cout << "Yes" << '\n';
    for (int eid : trail) { auto [u, v] = g.e[eid]; std::cout << u << ' '; }
    if (g.e.empty()) std::cout << 0 << '\n';
    else std::cout << g.e[trail.back()].v << '\n';
    for (int eid : trail) { eid >>= 1; std::cout << eid << ' '; }
    std::cout << '\n';
}
```

6 串

6.1 序列 Hash（随机 base，多 mod）

把整个序列的每个元素看成进制数的数位，序列开头的位权最大，结尾最小，便能得到序列的 Hash 值. 预处理每个前缀的 Hash 值 $h(i)$，则子段 $h[l,r) = h(r) - h(l) \cdot \text{base}^{r-l}$.

- 进制随机生成，用户可以指定 k 个模数，构造 `hasher`；
- `hasher.hash(s.begin(), s.end())` 可以预处理序列的 Hash 值，返回代理对象 `proxy`；
- `proxy.get(l, r)` 可以获取 `s[l, r)` k 个 Hash 值；
- 这 k 个 Hash 值使用 `std::vector` 存储，它可以直接进行字典序比较.

Code 40: seq-hash.cpp

```cpp
#include <chrono>
#include <random>

struct SeqHasher {

    struct Proxy {
        std::vector<u64> P;
        std::vector<std::vector<u64>> H, B;
        auto get(int l, int r) {
            std::vector<u64> A(H.size());
            for (int i = 0; i < H.size(); i++) {
                auto &a = A[i], &p = P[i];
                auto &h = H[i], &b = B[i];
                a = h[r] + (p - h[l]) * b[r - l];
                a %= p;
            }
            return A;
        }
    };

    template <class Iter>
    auto hash(Iter start, Iter end) {
        Proxy proxy;
        auto &[P, H, B] = proxy;

        P = mods;
        H.assign(mods.size(), {0});
        B.assign(mods.size(), {1});

        for (int k = 0; k < P.size(); k++) {
            int i = 0;
            auto &p = P[k]; auto &h = H[k], &b = B[k];
            for (auto it = start; it != end; i++) {
                h.push_back(h[i] * base + (*it++));
                b.push_back(b[i] * base);
                h.back() %= p;
                b.back() %= p;
            }
        }

        return proxy;
    }

    u64 base;
    std::vector<u64> mods;

    template <class ...U>
    SeqHasher(U ...mods) : mods{u64(mods)...} { set_base(); }

    void set_base() {
        auto is_prime = [](u64 n) {
            if (n <= 1) return false;
            for (u64 i = 2; i * i <= n; i++) if (n % i == 0) return false;
```

```
            return true;
        };
        auto tp = std::chrono::steady_clock::now();
        auto rng = std::mt19937_64(tp.time_since_epoch().count());
        base = rng() % 9982 + 1007;
        while (!is_prime(base)) base++;
    }

};

void test() {
    int n;
    std::cin >> n;
    std::vector<std::vector<u64>> a;
    SeqHasher hasher(998244353, 1e9 + 7, 1e9 + 9);
    for (int i = 0; i < n; i++) {
        std::string s;
        std::cin >> s;
        auto proxy = hasher.hash(s.begin(), s.end());
        auto hash = proxy.get(0, s.size());
        a.push_back(hash);
    }
    std::sort(a.begin(), a.end());
    std::cout << std::unique(a.begin(), a.end()) - a.begin() << '\n';
}
```

6.2 Manacher

Manacher 可以获取 s 每个字符为中心的最长回文子串，时间复杂度为 $O(n)$.

使用 s 字符集以外的 `sep` 字符，插空后得到 t. 比如 `mississippi` 变成 `#m#i#s#s#i#s#s#i#p#p#i#`，即 $s(i) \to t(2i+1)$. `manacher` 将返回 `r` 数组，代表以 $t(i)$ 为中心的最长回文半径 r（含中心）.

回文对应原串 $s\left[\left\lfloor\frac{i-r+1}{2}\right\rfloor, \left\lfloor\frac{i+r-1}{2}\right\rfloor\right)$，对应原串的回文长度为 $r-1$.

Code 41: manacher.cpp

```
template <class T, class Iter>
auto manacher(Iter start, Iter end, T sep) {
    std::vector<T> t; t.push_back(sep);
    while (start != end) t.push_back(*start++), t.push_back(sep);

    int n = t.size();
    std::vector<int> r(n);
    for (int i = 0, m = 0; i < n; i++) {
        int b = m + r[m];
        if (i < b) r[i] = std::min(r[m * 2 - i], b - i);
        int L = i - r[i], R = i + r[i];
        while (0 <= L && R < n && t[L] == t[R]) r[i]++, L--, R++;
        if (R > b) m = i;
    }

    return r;
}

void test() {
    std::string s;
    std::cin >> s;                                  // _m_i_s_s_i_s_s_i_p_p_i_
    auto r = manacher(s.begin(), s.end(), '_'); // 12121252181252121252121
    for (int i = 1; i < r.size() - 1; i++) {
        int L = (i - r[i] + 1) / 2;
        int R = (i + r[i] - 1) / 2;
        std::cout << s.substr(L, R - L) << '\n';
        assert(r[i] - 1 == R - L);
    }
}
```

6.3 KMP

kmp 获取每个前缀 $s[0,i)$ 的最长公共真前后缀的长度 $\pi(i)$，时间复杂度为 $O(n)$.

Code 42: kmp.cpp

```cpp
template <class Iter>
auto kmp(Iter s, Iter e) {
    std::vector<int> pi(e - s);
    for (int i = 1, k = 0; i < pi.size(); i++) {
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];
        if (s[i] == s[k]) k++;
        pi[i] = k;
    }
    return pi;
}

void test() {
    std::string s = "Hello Hello World Hello"; // Hello Hello World Hello
    auto pi = kmp(s.begin(), s.end());         // 00000012345600000012345
    for (auto &x : pi) std::cout << x << '\n';
}
```

6.4 Z 函数

zfunc 获取每个后缀 $s[i,n)$ 与原串 s 的最长公共前缀（LCP）的长度，时间复杂度为 $O(n)$.

Code 43: zfunc.cpp

```cpp
template <class Iter>
auto zfunc(Iter s, Iter e) {
    std::vector<int> z(e - s); z.front() = z.size();
    for (int i = 1, p = 0; i < z.size(); i++) {
        int b = p + z[p];
        z[i] = std::max(std::min(z[i - p], b - i), 0);
        while (i + z[i] < z.size() && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > b) p = i;
    }
    return z;
}

void test() {
    std::string s = "Hello Hello World Hello"; // Hello Hello World Hello
    auto z = zfunc(s.begin(), s.end());        // n0000060000000000050000
    for (auto &x : z) std::cout << x << '\n';
}
```

7 计算几何

1. **向量：**
   - 向量加减、数乘；
   - 向量的内积、外积；
   - 向量的模长、单位向量；
   - 向量的 $\pm\pi/2$ 旋转、θ 旋转；
   - 向量的夹角、$[-\pi,\pi]$ 有向角、$[0,2\pi]$ 有向角；
2. **点：**
   - 点所处半平面的符号；
   - 两点之间的重合判定；
   - 两点之间的 Eculid 距离；
   - 三点之间的逆时针/顺时针关系判定；
   - 三点之间的共线判定；
   - 点是否在角内的判定；
   - 点的字典序、方向序、极角序；
   - 点绕中心的 $\pm\pi/2$ 旋转、θ 旋转；
3. **直线：**
   - 直线的定义 $v\times P=c$；
   - 直线的两点式、点向式、点斜式、一般式；
   - 直线的平移构造、平行构造、垂直构造；
4. **点与直线：**
   - 点是否在直线上的判定；
   - 点到直线的带符号半平面距离、距离、带符号距离；
   - 点到直线的垂足、对称点；
5. **直线与直线：**
   - 两条直线的重合、平行、垂直、相交判定；
   - 两条直线的交点；
6. **点与线段：**
   - 点是否在直径确定的圆内的判定；
   - 点是否在线段上的判定；
   - 点到线段的距离；
7. **线段与线段：**
   - 两条线段完美相交的判定；
   - 两条线段的交点、交线段；
   - 两条线段之间的距离；
8. **圆：**
   - 圆的三点构造；
   - 圆上取点；
   - 过圆外一点作圆的切线；
   - 直线与圆的交点；
   - 圆与圆的交点；
9. **多边形：**
   - 多边形的面积；

- 点是否在多边形上/内的判定；
- 点是否在凸多边形内的判定；
- 点集的凸包；

10. **其他：**
    - 点集内的最近点对；

Code 44: geo2d.cpp

```cpp
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <vector>

using Real = long double;

constexpr Real EPS = 1e-8;
constexpr Real  PI = 3.14159265358979323846264338327950288419716939937510L;

bool lt(Real x, Real y)                                          { return x < y - EPS; }
bool le(Real x, Real y)                                          { return x <= y + EPS; }
bool gt(Real x, Real y)                                             { return lt(y, x); }
bool ge(Real x, Real y)                                             { return le(y, x); }
bool eq(Real x, Real y)                                    { return le(x, y) && ge(x, y); }
int sgn(Real x)                                             { return gt(x, 0) - lt(x, 0); }

struct Vec { Real x, y; };

Vec  operator-(Vec v)                                               { return {-v.x, -v.y}; }
Vec  operator+(Vec u, Vec v)                                { return {u.x + v.x, u.y + v.y}; }
Vec  operator-(Vec u, Vec v)                                { return {u.x - v.x, u.y - v.y}; }
Vec  operator*(Vec v, Real k)                                  { return {v.x * k, v.y * k}; }
Vec  operator*(Real k, Vec v)                                  { return {k * v.x, k * v.y}; }
Vec  operator/(Vec v, Real k)                                  { return {v.x / k, v.y / k}; }

Real operator*(Vec u, Vec v)                                { return u.x * v.x + u.y * v.y; }
Real operator^(Vec u, Vec v)                                { return u.x * v.y - u.y * v.x; }

std::istream& operator>>(std::istream& is, Vec& v)                { return is >> v.x >> v.y; }
std::ostream& operator<<(std::ostream& os, Vec  v) { return os<<'('<<v.x<<','<<v.y<<')'; }

Real len(Vec v)                                                 { return std::sqrtl(v * v); }
Vec  norm(Vec v)                             { Real r = len(v); return {v.x / r, v.y / r}; }

Vec  rotl(Vec v)                                                   { return {-v.y, v.x}; }
Vec  rotr(Vec v)                                                   { return {v.y, -v.x}; }
Vec  rot(Real th)                                 { return {std::cosl(th), std::sinl(th)}; }
Vec  rot(Real r, Real th)                    { return {r * std::cosl(th), r * std::sinl(th)}; }
Vec  rot(Vec v, Real th)               { return {v.x * std::cosl(th) - v.y * std::sinl(th),
                                                 v.x * std::sinl(th) + v.y * std::cosl(th)}; }
Real angle(Vec u, Vec v)                    { return std::fabsl(std::atan2l(u ^ v, u * v)); }
Real angle1(Vec u, Vec v)                                { return std::atan2l(u ^ v, u * v); }
Real angle2(Vec u, Vec v)    { Real th = angle1(u, v); return th < 0 ? th + PI / 2 : th; }

using Dot = Vec;

int  sgn(Dot p)                       { return (p.y > 0 || p.y == 0 && p.x > 0) ? 1 : -1; }
Real dist(Dot a, Dot b)                                             { return len(a - b); }
Real orient(Dot o, Dot a, Dot p)                                { return (a - o) ^ (p - o); }

Dot  rotl(Dot p, Dot o)                                           { return o + rotl(p - o); }
Dot  rotr(Dot p, Dot o)                                           { return o + rotr(p - o); }
Dot  rot(Dot p, Dot o, Real th)                                { return o + rot(p - o, th); }

bool is_same(Dot a, Dot b)                              { return eq(a.x, b.x) && eq(a.y, b.y); }
bool is_collinear(Dot o, Dot a, Dot b)                       { return eq(orient(o, a, b), 0); }
bool in_angle(Dot o, Dot a, Dot b, Dot p)    { if (orient(o, a, b) < 0) std::swap(a, b);
```

```
60                                  return ge(orient(o, a, p), 0) && le(orient(o, b, p), 0); }
61
62  struct lex_order {
63      bool y_prior = false;
64      bool operator()(Dot a, Dot b) {
65          if (y_prior) return a.y == b.y ? a.x < b.x : a.y < b.y;
66          return a.x == b.x ? a.y < b.y : a.x < b.x;
67      }
68  };
69
70  struct direc_order {
71      Vec v = {1, 0};
72      bool operator()(Dot a, Dot b) {
73          return a * v < b * v;
74      }
75  };
76
77  struct polar_order {
78      Dot o = Dot{0, 0};
79      bool operator()(Dot a, Dot b) {
80          a = a - o, b = b - o;
81          return std::tuple{sgn(a),     0, a * a} <
82                 std::tuple{sgn(b), a ^ b, b * b};
83      }
84  };
85
86  struct Line { Vec v; Real c; };
87  struct Segm { Dot a, b; };
88
89  Line line(Dot a, Dot b)                                 { Vec v = b - a; return {v, v ^ a}; }
90  Line line_pv(Dot p, Vec v)                                              { return {v, v ^ p}; }
91  Line line_th(Dot p, Real th)                                  { return line_pv(p, rot(th)); }
92  Line line_abc(Real a, Real b, Real c)                       { return {rotr(Vec{a, b}), c}; }
93  Line line_seg(Segm s)                                             { return line(s.a, s.b); }
94
95  Line shift(Line l, Vec v)                                   { return {l.v, l.c + (l.v ^ v)}; }
96  Line para(Dot p, Line l)                                          { return line_pv(p, l.v); }
97  Line perp(Dot p, Line l)                                   { return line_pv(p, rotl(l.v)); }
98
99  Real side(Dot p, Line l)                                              { return (l.v ^ p) - l.c; }
100 Real dist(Dot p, Line l)                          { return std::fabsl(side(p, l)) / len(l.v); }
101 Real dist1(Dot p, Line l)                                  { return side(p, l) / len(l.v); }
102
103 Dot  proj(Dot p, Line l)              { return p - rotl(l.v) * side(p, l) / (l.v * l.v); }
104 Dot  refl(Dot p, Line l)          { return p - rotl(l.v) * 2 * side(p, l) / (l.v * l.v); }
105 Dot  itsc(Line m, Line n)                { return (m.c * n.v - n.c * m.v) / (m.v ^ n.v); }
106
107 std::ostream& operator<<(std::ostream &os, Line l)
108                     { return os << '[' << proj(Dot{0, 0}, l) << ',' << norm(l.v) << ']'; }
109 std::ostream& operator<<(std::ostream &os, Segm s) { return os<<'['<<s.a<<','<<s.b<<']'; }
110
111 bool on_line(Dot p, Line l)                                       { return eq(side(p, l), 0); }
112 bool is_same(Line m, Line n)                  { return eq(m.v ^ n.v, 0) && eq(m.c, n.c); }
113 bool is_para(Line m, Line n)                 { return eq(m.v ^ n.v, 0) && !eq(m.c, n.c); }
114 bool is_perp(Line m, Line n)                                    { return eq(m.v * n.v, 0); }
115 bool is_itsc(Line m, Line n)                                   { return !eq(m.v ^ n.v, 0); }
116
117 bool in_disk(Dot p, Segm s)                          { return le((s.a - p) * (s.b - p), 0); }
118 bool on_segm(Dot p, Segm s)          { return is_collinear(p, s.a, s.b) && in_disk(p, s); }
119
120 auto itsc(Segm m, Segm n) {
121     std::vector<Dot> ans;
122     Real oa = orient(n.a, n.b, m.a), ob = orient(n.a, n.b, m.b);
123     Real oc = orient(m.a, m.b, n.a), od = orient(m.a, m.b, n.b);
124     if (oa * ob < 0 && oc * od < 0) {
125         ans.push_back((m.a * ob - m.b * oa) / (ob - oa));
126         return ans;
127     }
128     if (on_segm(m.a, n)) ans.push_back(m.a);
```

```
    if (on_segm(m.b, n)) ans.push_back(m.b);
    if (on_segm(n.a, m)) ans.push_back(n.a);
    if (on_segm(n.b, m)) ans.push_back(n.b);
    return ans;
}

bool is_perfect_itsc(Segm m, Segm n) {
    Real oa = orient(n.a, n.b, m.a), ob = orient(n.a, n.b, m.b);
    Real oc = orient(m.a, m.b, n.a), od = orient(m.a, m.b, n.b);
    return oa * ob < 0 && oc * od < 0;
}

Real dist(Dot p, Segm s) {
    if (is_same(s.a, s.b)) return len(s.a - p);
    Line l = line_seg(s);
    direc_order cmp{l.v};
    if (cmp(s.a, p) && cmp(p, s.b)) return dist(p, l);
    return std::min(len(s.a - p), len(s.b - p));
}

Real dist(Segm m, Segm n) {
    if (is_perfect_itsc(m, n)) return 0;
    return std::min({
        dist(m.a, n), dist(m.b, n),
        dist(n.a, m), dist(n.b, m)
    });
}

struct Circ { Dot o; Real r; };

Circ circ(Dot a, Dot b, Dot c) {
    b = b - a, c = c - a;
    Dot o = a + itsc(line_pv(b / 2, rotl(b)), line_pv(c / 2, rotl(c)));
    return {o, len(a - o)};
}

Dot  circ_at(Dot o, Real r, Real th)                          { return o + rot(r, th); }
Dot  circ_at(Circ O, Real th)                              { return circ_at(O.o, O.r, th); }

auto tangent(Dot o, Real r, Dot p) {
    Vec po = o - p;
    Real d = len(po);
    std::vector<Line> ans;
    if (lt(d, r)) return ans;
    if (eq(d, r)) return ans.push_back(line_pv(p, rotr(po))), ans;
    Real th = std::asinl(r / d);
    ans.push_back(line_pv(p, rot(po, th)));
    ans.push_back(line_pv(p, rot(po, -th)));
    return ans;
}

auto itsc(Dot o, Real r, Line l) {
    std::vector<Dot> ans;
    Real d = dist(o, l);
    Real sqh = r * r - d * d;
    if (lt(sqh, 0)) return ans;
    Dot m = proj(o, l);
    if (eq(sqh, 0)) return ans.push_back(m), ans;
    Vec h = norm(l.v) * std::sqrtl(sqh);
    ans.push_back(m + h), ans.push_back(m - h);
    return ans;
}

auto itsc(Dot o1, Real r1, Dot o2, Real r2) {
    std::vector<Dot> ans;
    Vec o1o2 = o2 - o1;
    Real D = len(o1o2);

    if (gt(D, r1 + r2)) return ans;
```

```
    if (lt(D, std::fabsl(r1 - r2))) return ans;
    if (eq(D, r1 + r2) || eq(D, std::fabsl(r1 - r2))) {
        ans.push_back(o1 + norm(o1o2) * r1);
        return ans;
    }

    Real d = (D * D + r1 * r1 - r2 * r2) / (2 * D);
    Real h = std::sqrtl(r1 * r1 - d * d);
    Dot m = o1 + norm(o1o2) * d;
    Vec v = norm(rotl(o1o2)) * h;
    ans.push_back(m + v);
    ans.push_back(m - v);
    return ans;
}

auto tangent(Circ O, Dot p)                              { return tangent(O.o, O.r, p); }
auto itsc(Circ O, Line l)                                  { return itsc(O.o, O.r, l); }
auto itsc(Circ O1, Circ O2)                        { return itsc(O1.o, O1.r, O2.o, O2.r); }

using Polygon = std::vector<Dot>;
using    Poly = Polygon const &;

Real area1(Poly poly) {
    Real ans = 0;
    int n = poly.size();
    for (int i = 0; i < n - 1; i++) ans += poly[i] ^ poly[i + 1];
    ans += poly[n - 1] ^ poly[0];
    return ans / 2.0;
}

Real area(Poly poly) {
    return std::fabsl(area1(poly));
}

bool on_poly(Dot o, Poly poly) {
    for (int i = 0, j = 1; i < poly.size(); i++, j++) {
        if (j >= poly.size()) j -= poly.size();
        if (on_segm(o, Segm{poly[i], poly[j]})) return true;
    }
    return false;
}

bool in_poly(Dot o, Poly poly, bool strict) {
    if (on_poly(o, poly)) return !strict;
    auto cross_test = [o] (Dot a, Dot b) {
        if (orient(o, a, b) > 0) return a.x <= o.x && o.x < b.x;
        return b.x <= o.x && o.x < a.x;
    };
    bool flag = false;
    for (int i = 0, j = 1; i < poly.size(); i++, j++) {
        if (j >= poly.size()) j -= poly.size();
        flag ^= cross_test(poly[i], poly[j]);
    }
    return flag;
}

bool in_convex(Dot o, Poly poly, bool strict) {
    int n = poly.size();
    if (lt(orient(poly[0], poly[1], o), 0)) return false;
    if (lt(orient(poly[n-1], poly[0], o), 0)) return false;
    if (on_segm(o, Segm{poly[0], poly[1]})) return !strict;
    if (on_segm(o, Segm{poly[n-1], poly[0]})) return !strict;
    int l = 1, r = n - 2;
    while (l < r) {
        int m = (l + r + 1) / 2;
        if (gt(orient(poly[0], poly[m], o), 0)) l = m;
        else r = m - 1;
    }
    if (on_segm(o, Segm{poly[l], poly[l+1]})) return !strict;
```

```
    return gt(orient(poly[l], poly[l+1], o), 0);
}

auto hull(std::vector<Dot> dots) {
    if (dots.size() <= 1) return dots;
    std::vector<Dot> lo, hi;
    std::sort(dots.begin(), dots.end(), lex_order{});
    for (Dot p : dots) {
        while (lo.size() > 1 && ge((p - lo.end()[-1]) ^ (p - lo.end()[-2]), 0))
            lo.pop_back();
        while (hi.size() > 1 && le((p - hi.end()[-1]) ^ (p - hi.end()[-2]), 0))
            hi.pop_back();
        lo.push_back(p);
        hi.push_back(p);
    }
    std::reverse(hi.begin(), hi.end());
    lo.pop_back(), hi.pop_back();
    lo.insert(lo.end(), hi.begin(), hi.end());
    return lo;
}

struct closest_pair_answer { Real sqd; Dot u, v; };
auto closest_pair(std::vector<Dot> dots) {

    constexpr Real INF = INT64_MAX;
    using answer = closest_pair_answer;

    auto solve = [&](auto &f, int l, int r) -> answer {

        if (l + 1 == r) return {INF, {INF, INF}, {INF, INF}};

        int m = (l + r) / 2;
        Real midx = dots[m].x;
        auto ansl = f(f, l, m);
        auto ansr = f(f, m, r);

        Real sqd; Dot u, v;
        if (ansl.sqd <= ansr.sqd) sqd = ansl.sqd, u = ansl.u, v = ansl.v;
        else sqd = ansr.sqd, u = ansr.u, v = ansr.v;
        std::inplace_merge(dots.begin() + l, dots.begin() + m,
                           dots.begin() + r, lex_order{true});

        std::vector<int> cand;
        cand.reserve(r - l);
        for (int i = l; i < r; i++) {
            Real dx = dots[i].x - midx;
            if (dx * dx < sqd) cand.push_back(i);
        }

        for (int i = 0; i < cand.size(); i++) {
            for (int j = i + 1; j < cand.size(); j++) {
                Real dy = dots[cand[j]].y - dots[cand[i]].y;
                if (dy * dy >= sqd) break;
                Real dx = dots[cand[j]].x - dots[cand[i]].x;
                Real ds = dx * dx + dy * dy;
                if (ds < sqd) sqd = ds, u = dots[cand[i]], v = dots[cand[j]];
            }
        }

        return {sqd, u, v};
    };

    std::sort(dots.begin(), dots.end(), lex_order{});
    return solve(solve, 0, dots.size());

}
```

8 其他

8.1 整数区间二分

整数区间 $[L, R)$ 满足单调性：NNNYYYY 或 YYYYNNN，其中 Y 表示成功，N 表示失败.
获取 NNNYYYY 的第一个 Y：判断 $[L, M)$ 的最后一个元素是否为 Y.

Code 45: binary-search-first.cpp

```cpp
void test() {
    int L = -114, R = 514;       // [L, R)
    while (R - L > 1) {
        int M = (L + R) / 2;
        if (judge(M - 1)) R = M; // [L, M)
        else L = M;              // [M, R)
    }
    if (judge(L)) std::cout << "NO" << '\n';
    else std::cout << L << '\n';
}
```

获取 YYYYNNN 的最后一个 Y：判断 $[M, R)$ 的第一个元素是否为 Y.

Code 46: binary-search-last.cpp

```cpp
void test() {
    int L = -114, R = 514;       // [L, R)
    while (R - L > 1) {
        int M = (L + R) / 2;
        if (judge(M)) L = M;     // [M, R)
        else R = M;              // [L, M)
    }
    if (judge(L)) std::cout << "NO" << '\n';
    else std::cout << L << '\n';
}
```

8.2 单峰离散函数最值区间

可以使用二分来获取形如 abdEEEEca 单峰函数的最值区间，前提是这个单峰函数满足：

1. 所有最值点构成整数区间 $[l_m, r_m)$；
2. 函数在 $[L, l_m + 1)$ 和 $[r_m - 1, R)$ 上严格单调且调性相反.

以最大值（a<b< < = = = > >，在 $[L + 1, R)$ 上二分搜索）为例：

Code 47: single-mount.cpp

```cpp
template <class Int, class J>
Int find_first_if(Int L, Int R, J judge) {
    while (R - L > 1) {
        Int M = (L + R) / 2;
        if (judge(M - 1)) R = M;
        else L = M;
    }
    return judge(L) ? L : L + 1;
}

template <class Int, class J>
Int find_last_if(Int L, Int R, J judge) {
    while (R - L > 1) {
        Int M = (L + R) / 2;
        if (judge(M)) L = M;
        else R = M;
    }
    return judge(L) ? L : L - 1;
```

```
}

int main() {
    // index:  0, 1, 2, 3, 4, 5, 6, 7
    int a[] = {2, 2, 2, 2, 2, 2, 2, 2};
    int b[] = {2, 4, 5, 7, 7, 7, 3, 1};
    int L, R;
    L = find_last_if(1, 8, [&](int i) { return a[i - 1] < a[i]; });
    R = find_first_if(1, 8, [&](int i) { return a[i - 1] > a[i]; });
    assert(L == 0 && R == 8);
    L = find_last_if(1, 8, [&](int i) { return b[i - 1] < b[i]; });
    R = find_first_if(1, 8, [&](int i) { return b[i - 1] > b[i]; });
    assert(L == 3 && R == 6);
    std::cout << "ok" << '\n';
}
```

8.3 std::lower_bound / std::upper_bound

1. bound 就是区间端点，左闭右开；
2. std::lower_bound 支持 bool(item, value)，返回第一个结果为假的 item 索引；
3. std::upper_bound 支持 bool(value, item)，返回第一个结果为真的 item 索引.

Code 48: range-bound.cpp

```
void test() {

    // index:  0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
    int a[] = {1, 2, 4, 4, 4, 4, 4, 9, 9, 9, 11, 13, 14};
    int *L = std::begin(a), *R = std::end(a);

    int *l, *r;
    l = std::lower_bound(L, R, 9);
    r = std::upper_bound(L, R, 9); // [7, 10)
    l = std::lower_bound(L, R, 8);
    r = std::upper_bound(L, R, 8); // [7, 7)

    int x = 4, *p1, *p2, *p3, *p4;
    p1 = std::lower_bound(L, R, x);     // first of >= x
    p2 = std::upper_bound(L, R, x);     // first of >  x
    p3 = std::lower_bound(L, R, x) - 1; //  last of <  x
    p4 = std::upper_bound(L, R, x) - 1; //  last of <= x

    assert(p1 - a == 2 && p2 - a == 7);
    assert(p3 - a == 1 && p4 - a == 6);

}
```

8.4 std::partial_sum / std::adjacent_difference

查询和修改左闭右开区间，先往前缀和数组头部添加幺元再求前缀和，先往原数组尾部添加幺元再求差分.

Code 49: sum-diff.cpp

```
void test() {

    std::vector<int> a{2, -3, 4, 1, -5};

    a.emplace_back();
    std::adjacent_difference(a.begin(), a.end(), a.begin());
    a[2] += 10, a[5] -= 10; // a[2, 5) += 10
    std::partial_sum(a.begin(), a.end(), a.begin());
    a.pop_back();
    assert((a == std::vector{2, -3, 14, 11, 5}));
```

```
11
12      std::vector<int> p{0};
13      std::partial_sum(a.begin(), a.end(), std::back_inserter(p));
14      assert(p[4] - p[1] == 22); // sum a[1, 4)
15
16  }
```

8.5 std::next_permutation / std::prev_permutation

实现的基本思路是：xxx 3 75421 末尾的最长逆序段 75421 不存在下一个字典序，而 3 75421 存在下一个字典序，且首位必为逆序段大于 3 的最小值 4，这样便得到 4 75321，末尾 75321 保持逆序，逆回正序变得到最小字典序 12357，于是结果为 xxx 4 12357.

- 时间复杂度最好为 $O(1)$（接近顺序），最坏为 $O(n)$（接近逆序）；
- 当完全逆序时，std::next_permutation 将返回 false，并变回正序.

Code 50: permutation.cpp

```
1  void test() {
2      int a[] = {0, 1, 2, 3, 4};
3      do { for (auto x : a) std::cout << x << ' '; std::cout << '\n'; }
4      while (std::next_permutation(a, a + 5));
5      assert(std::equal(a, a + 5, (int[]){0, 1, 2, 3, 4}));
6  }
```

8.6 自定义 Hash

需要自定义 hasher 仿函数，以及重载 operator==.
使用 unordered 类容器，如果需要防 Hack，常数会很大.
如果有 libstdc++ 库，可以使用 __gnu_pbds::gp_hash_table，
用 find(x) != end() 来代替 count()，其他接口与之前相同.

Code 51: custom-hash.cpp

```
1  #include <chrono>
2  #include <functional>
3  #include <ext/pb_ds/assoc_container.hpp>
4
5  struct A {
6      int x; double y;
7      bool operator==(A const &o) const { return x == o.x && y == o.y; };
8  };
9
10 struct B {
11     A a; bool z;
12     bool operator==(B const &o) const { return a == o.a && z == o.z; };
13 };
14
15 struct Hasher {
16
17     static u64 magic(u64 x) {
18         x += 0x9e3779b97f4a7c15;
19         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
20         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
21         return x ^ (x >> 31);
22     }
23
24     // default
25     template <class T>
26     u64 operator()(T const &x) const {
27         static const auto tp = std::chrono::steady_clock::now();
```

```
28          static const u64 eps = tp.time_since_epoch().count();
29          return magic(std::hash<T>()(x) + eps);
30      }
31
32      template <class ...T>
33      u64 mix(T ...x) const { return ((*this)(x) ^ ...); }
34
35      // pair
36      template <class T, class U>
37      u64 operator()(std::pair<T, U> const &p) const { return mix(p.first, p.second); }
38
39      // tuple
40      template <class ...T>
41      u64 operator()(std::tuple<T...> const &t) const
42      { return std::apply([this](T const &...x) { return mix(x...); }, t); }
43
44      // vector
45      template <class T>
46      u64 operator()(std::vector<T> const &v) const {
47          u64 result = 0;
48          for (auto const &x : v) result ^= (*this)(x);
49          return result;
50      }
51
52      // user
53      u64 operator()(A const &a) const { return mix(a.x, a.y); }
54      u64 operator()(B const &b) const { return mix(b.a, b.z); }
55
56  };
57
58  template<class S, class T>
59  using umap = __gnu_pbds::gp_hash_table<S, T, Hasher>;
60
61  int main() {
62      std::unordered_set<std::tuple<int, double, std::string>, Hasher> S0;
63      std::unordered_set<std::pair<A, B>, Hasher> S1;
64      umap<B, int> S2;
65
66      S1.insert({{8, 1.0}, {}});
67      S1.insert({{7, 1.0}, {{}, true}});
68      S1.insert({{8, 1.0}, {}});
69      S1.insert({{6, 1.0}, {}});
70      S1.insert({{7, 1.0}, {{}, true}});
71
72      assert(S1.size() == 3);
73  }
```

8.7 使用栈模拟递归

程序是一台状态机：

- 栈帧：PC + 函数参数 + 局部变量；
- 状态：栈 + 全局变量；
- 转移：语句.

我们可以维护一台状态机：

1. 以递归语句、返回语句为界，划分代码块；
2. 每个代码块被分配 PC 值；
3. 调用递归时，往栈压入栈帧；
4. 结束返回时，先把结果放在 `rax` 中，然后弹栈.

当然完全模拟栈帧来写会比较繁琐，实际根据递归的复杂程度进行一定的简化.

Code 52: stimulate-recursive.cpp

```cpp
int hanoi(int n, char from, char via, char to) {

    struct Frame { int pc; int n; char from, via, to; int t; };
    std::stack<Frame> stk;
    int rax = 0;

    stk.push({0, n, from, via, to, 0});
    while (!stk.empty()) {
        auto &[pc, n, from, via, to, t] = stk.top();
        bool flag = false;

        if (pc == 0) {
            if (n == 1) {
                std::cout << from << "->" << to << '\n';
                rax = 1, stk.pop(), flag = true;
            }
        }

        else if (pc == 1) stk.push({0, n - 1, from, to, via, 0});
        else if (pc == 2) t += rax;
        else if (pc == 3) std::cout << from << "->" << to << '\n', t++;
        else if (pc == 4) stk.push({0, n - 1, via, from, to, 0});
        else if (pc == 5) rax = t + rax, stk.pop(), flag = true;

        if (!flag) pc++;
    }

    return rax;

}

// int hanoi(int n, char from, char via, char to) {
//     if (n == 1) return std::cout << from << "->" << to << '\n', 1;
//     int t = 0;
//     t += hanoi(n - 1, from, to, via);
//     std::cout << from << "->" << to << '\n'; t++;
//     t += hanoi(n - 1, via, from, to);
//     return t;
// }
```

8.8 位运算操作

最高位：

- $\lfloor \log_2 x \rfloor$：`31 - __builtin_clz(x)`；
- $\lceil \log_2 x \rceil$：`32 - __builtin_clz(x - 1)`；
- 最高位 1 的位置：`31 - __builtin_clz(x)`.

最低位：

- 最低位 1 的位置：`__builtin_ctz(x)`；
- 清除最低位的 1：`x &= x - 1`；
- 取最低位的 1：`x & -x`.

第 i 位：

- 查看第 i 位：`x >> i & 1`；
- 第 i 位置 1：`x |= 1 << i`；
- 第 i 位置 0：`x &= ~(1 << i)`；
- 第 i 位取反：`x ^= 1 << i`.

2 的幂：

- 判断是否为 2 的幂：`x > 0 && (x & (x - 1) == 0)`；
- 离 $x > 0$ 最近的 2^k：$2^{\lfloor \log_2 x \rfloor} \le x \le 2^{\lceil \log_2 x \rceil}$.

掩码：

- $[l, h)$：`((1 << h) - 1) ^ ((1 << l) - 1)`；
- 偶数位：`0x55555555`；
- 奇数位：`0xAAAAAAAA`.

其他：

- 1 的个数：`__builtin_popcount(x)`；
- 原码转 Gray 码：`g = x ^ (x >> 1)`；
- Gray 码转原码：`for (x = 0; g; g >>= 1) x ^= g`；
- 子集枚举：`for (int sub = x; sub; sub = (sub - 1) & x)`；
- 含 k 个 1 的 n 位数枚举：

Code 53: enum-k-1s.cpp

```cpp
void test() {
    int k = 5, n = 8;
    // 00011111 00101111 00110111 ... 11111000
    for (int s = (1 << k) - 1, t; s < (1 << n);) {
        std::cout << std::bitset<8>(s) << '\n';
        t = s + (s & -s);
        s = (s & ~t) >> (__builtin_ctz(s & -s) + 1) | t;
    }
}
```